

Algorithmische Morphologien für autonome Roboter

vorgelegt von

Vincent Rist

dem Fachbereich VII
Elektrotechnik – Mechatronik – Optometrie
der Berliner Hochschule für Technik
zur Erlangung des akademischen Grades

Bachelor of Engineering (B.Eng.)

im Studiengang

Humanoide Robotik

Tag der Abgabe: 1. Februar 2022

Gutachter

Prof. Dr. rer. nat. M. Hild Berliner Hochschule für Technik
B. Panreck Berliner Hochschule für Technik



Kurzfassung

Diese Arbeit beschreibt einen Gestaltungsprozess für autonome Roboter, inklusive Entwurf, Analyse und Einsatz. Kreisbögen dienen als geometrisches Werkzeug, um zweidimensionale Roboter mit runden, geschmeidigen Körperformen zu modellieren. Aus diesen Robotermodellen wird eine morphologische Mannigfaltigkeit generiert, welche als gerichteter Graphen gespeichert und nach Bewegungsabläufen durchsucht werden kann. Ein 3D-druckbares Steckkit ermöglicht eine schnelle Inbetriebnahme und Evaluation Morphologien und Bewegungen. Ich teste den Prozess am Beispiel der Fortbewegung. Algorithmisch ermittelte Schlüsselposen bringen Roboter mit verschiedenen morphologischen Merkmalen dazu, sich stabil vorwärtszubewegen.

Abstract

This thesis describes a process for designing autonomous robots, including draft, analysis, and deployment. Circular arcs serve as a tool, that is able to model robots with smooth, round body shapes. A morphological manifold generated from these models can in turn be saved as a directed graph and be searched for motion sequences. Morphology and behavior can quickly be deployed and evaluated with a 3D-printable plug-n-clamp kit. I test the whole process via the exemplary task of locomotion. Algorithmically discovered key postures enable robots with different morphological features to move forward in a robust manner.

Danksagung

Ich bedanke mich bei Veronica Eirich und Leon Kollofrath für ihre Hilfe und Inspiration beim Gestalten des LARA-Steckkits. Im besonderen Maße bedanke ich mich bei Joshua Göttlich, der mir bei der Einarbeitung in die Software für die Motoren assistiert hat. Außerdem möchte ich für die Unterstützung meiner Betreuer Benjamin Panreck und Manfred Hild sowie beim restlichen Team des NRL Forschungslabors meinen Dank aussprechen.

Selbstständigkeitserklärung

Hiermit gebe ich eine eidesstattliche Erklärung ab, dass ich die vorliegende Arbeit selbstständig verfasst habe. Sämtliche Stellen der Arbeit, die im Wortlaut oder dem Sinn nach Publikationen oder Vorträgen anderer Autoren entnommen sind, habe ich als solche kenntlich gemacht.

Vincent Rist

Name, Unterschrift

A handwritten signature in black ink, appearing to read 'Vincent Rist', written over a horizontal line.

Berlin, 30. Januar 2022

Ort, Datum

Inhalt

1	Einleitung	1
1.1	Begriffsklärungen	2
1.2	Zielsetzung und Vorgehensweise	4
2	Stand der Forschung	5
3	Grundlagen	9
3.1	Roboterposen	9
3.2	Dynamische Systeme	11
3.3	Morphologische Mannigfaltigkeiten	14
3.3.1	Mehr Dimensionen	15
3.3.2	Interpretation der Mannigfaltigkeit	17
3.4	Graphentheorie	18
3.4.1	Mannigfaltigkeit als Graph	19
4	Konzept	21
4.1	Darstellung eines Roboters	22
4.1.1	Roboter-Datenstruktur	22
4.1.2	Kreisbögen zum Modellieren von Körperformen	23
4.1.3	Kreisbogenformen im Vergleich mit Polygonen	25
4.2	Generierung der Mannigfaltigkeit	26
4.2.1	Massenschwer- und Auflagepunkte einer Pose	27
4.2.2	Stabile Posen als Knoten speichern	31
4.2.3	Nachbarn mit Kanten verbinden	32
4.2.4	Zuordnung einer Untermannigfaltigkeit	35
4.3	Suche nach Bewegungsabläufen	37
4.4	Robotersteckkit	43

5	Evaluation	46
5.1	Versuchsaufbau	47
5.2	Versuchsreihe 1	49
5.3	Versuchsreihe 2	51
6	Fazit	55
6.1	Zusammenfassung	56
6.2	Weiterführende Arbeit	58
A	Anhang: Modellieren durch Kreisbögen	60
A.1	Mathematische Definition	60
A.1.1	Kreisbogenform	63
A.1.2	Erweiterung für Ecken	65
A.2	Cashew-Form	66
	Abbildungsverzeichnis	71
	Tabellenverzeichnis	72
	Algorithmusverzeichnis	72
	Literatur	73

1 Einleitung

„By embodiment, we mean that intelligence always requires a body. Or, more precisely, we ascribe intelligence only to agents that are embodied, i.e., real physical systems whose behavior can be observed as they interact with the environment.“

R. Pfeifer und J. Bongard in [1, Seite 18]

Das Zitat skizziert die Theorie, dass die Entwicklung von Intelligenz immer einen Körper benötigt und dass sie sich durch diese Verkörperung prozesshaft und interaktiv entwickelt. Diese Idee der Intelligenz wird als *Embodiment* betitelt. Nicht das Gehirn, sondern der Körper bilde die Grundlage, um intelligentes Verhalten auf die Beine zu stellen. Software allein, wie beispielsweise die häufig verwendeten neuronalen Netze, die bereits anspruchsvolle Aufgaben wie Bilderkennung oder Sprachverständnis bewältigen, besitzen keine Körper und könnten demnach auch keine Intelligenz entwickeln. Für Roboter, die selbstständig und ohne menschlichen Einfluss agieren, sieht das jedoch anders aus. Der Prozessor – das Roboterhirn – bekommt Sensorstimulationen von realen, physikalischen Größen wie Licht, Schall oder Schwerkraft und muss diese in Motoransteuerungen umwandeln, welche einen direkten Einfluss auf ihre Umwelt nehmen. Die Umwelt der Roboter folgt zwar regelmäßigen, aber dennoch nicht immer eindeutigen physikalischen Gesetzen. Der Körper bildet folglich das Bindeglied zwischen Gehirn und Umgebung und beeinflusst laut *Embodiment* zu hohem Maße, wie sich die Intelligenz auf dem verkörperten System entwickeln kann.

Es stellt sich also die Frage: Wie müsste ein Gestaltungsprozess für autonomen Robotern aussehen, der die Eigenschaften der Morphologie, konkreter morphologischer Mannigfaltigkeiten, berücksichtigt und als Grundlage für verhaltenssteuernde Algorithmen nutzt?

Nehmen wir diese Frage erst einmal auseinander.

1.1 Begriffsklärungen

In der Biologie beschreibt *Morphologie* die Form oder Gestalt von Lebewesen. Das beinhaltet die Anordnung von Körperteilen und Organen (Anatomie) und das äußerliche Erscheinungsbild (Eidonomie), welches durch Farbe, Größe und Form geprägt wird. Aus der sprachwissenschaftlichen Perspektive erforscht die Morphologie, aus welchem Stamm ein Wort sich bildet und welche Wortarten und Flexionsformen es besitzt. „Metamorphose“ beschreibt einen biologischen Gestaltwandel, wie zum Beispiel die Transformation einer Raupe zu einem Schmetterling und unter „anthropomorph“ versteht man die Ähnlichkeit zum Menschen. Der Begriff kann hier im Zusammenhang mit der geometrischen Körperform eines Roboters und der räumlichen Anordnung seiner Körperteile eingeordnet werden.

Auch die mathematische Morphologie beschäftigt sich mit geometrischen Strukturen beispielsweise in Bildern oder 3D-Modellen. Sie ist eng verwandt mit der Topologie, der Disziplin, dem der Begriff *Mannigfaltigkeit* entspringt. Dabei handelt es sich um eine Punktmenge, die lokal dem euklidischen Raum ähnelt, dennoch aber komplexere Formen annehmen kann. Das geläufigste Beispiel ist eine Kugel im dreidimensionalen Raum, wie unsere Erde. An jeder einzelnen Stelle lässt sich die Oberfläche durch eine zweidimensionale Landkarte abbilden, aber keine Fläche kann eindeutig jeden Punkt auf der Kugel definieren. Diese Mannigfaltigkeiten stellen sich als nützliches Werkzeug heraus, um den Zustandsraum von Roboterposen abzubilden.

Autonomie ist ein Synonym für Selbstständigkeit, Unabhängigkeit und Entscheidungsfreiheit. Besonders im Zusammenhang mit autonomem Fahren hat der Begriff an Aufmerksamkeit erlangt. Man versucht dabei, dem Menschen das Steuer zu entziehen und stattdessen das Fahrzeug mit einem Programm durch den Verkehr zu navigieren. Analog entscheiden autonome Roboter selbst über ihre Aktionen in der Welt, anstatt dass sie von Menschen handgesteuert werden.

Dabei ist die Frage, wie autonom ein Roboter sein kann. Ein vom Menschen ferngesteuerter Roboter hat nur wenig bis gar keine Autonomie. Industrieroboter, die einen einprogrammierten Prozess automatisieren, bewegen sich zwar autonom, aber treffen ihre Entscheidungen nicht selbst. Adaptive Lernverfahren können sich hingegen an un-

1 EINLEITUNG

terschiedliche Gegebenheiten anpassen und auf Störungen reagieren, doch auch sie sind durch eine menschliche Hand entstanden. Gilt ein Algorithmus also erst als autonom, wenn er durch einen Mutteralgorithmus erschaffen wurde? Es genügt wohl, sich darauf zu einigen, dass es verschiedene Abstufungen von Autonomie geben kann.

Ein Algorithmus ist eine Reihe von festen Anweisungen wie ein Rezept oder eine Bedienungsanleitung. Eine mögliche Anweisung ist die Bedingungsabfrage in der Form **if A then B else C**. Eine andere Möglichkeit sind Schleifen wie **while D do E**, die einen Befehl wiederholt ausführen. Algorithmen können dabei beliebig kompliziert sein, müssen aber durch eine endliche Anzahl von Anweisungen formuliert werden können. Es gibt unterschiedliche Arten von Algorithmen. Sie gelten als *determinierend*, wenn sie für dieselben Startbedingungen immer dasselbe Ergebnis liefern, und als *deterministisch* bezeichnet man den Algorithmus, wenn zu jedem Zeitpunkt und zu jeder Situation definiert ist, was der nächste Schritt zu sein hat. In der Mathematik und Informatik werden Algorithmen unter anderem zum Rechnen, zur Klassifikation oder zur Datenverarbeitung verwendet. Besonders in der Robotik können sie auch zum Steuern von Verhaltensweisen eingesetzt werden. Da sich Algorithmen eignen, um praktisch jede Tätigkeit als Liste von Anweisungen zu abstrahieren, formuliert auch diese Arbeit einige Algorithmen in Form von Pseudoquelltexten.

Roboter samt Morphologie und Programmierung müssen von Menschenhand entworfen werden, denn bis dato ist die Autonomie von Robotern zur Selbstproduktion noch nicht fortgeschritten genug. Robotische Systeme sind komplexe Angelegenheiten, an denen einige Disziplinen mitmischen wie zum Beispiel die Mechanik, die Elektronik und die Informatik, aber auch die Psychologie. Solche Systeme so flexibel zu gestalten, dass sie im Nachhinein noch physische Erweiterungen aufnehmen können, ist eine anspruchsvolle Aufgabe. Obwohl sie laut Embodiment eine große Rolle zur Entwicklung intelligenten Verhaltens spielt, kann sich die Morphologie eines fertigen Roboters kaum verändern und muss deswegen schon bei der Roboterentwicklung sorgfältig modelliert werden.

1.2 Zielsetzung und Vorgehensweise

In der Bachelorarbeit entsteht ein Gestaltungsprozess für autonome Roboter, welcher nach dem Prinzip von *Embodiment* die Morphologie und ihre Bedeutungen für die Handlungsmöglichkeiten des Roboters in den Vordergrund stellt. Folgende Ziele stehen dabei im Zentrum:

1. Das Definieren der Kreisbogendarstellung als geometrisches Werkzeuges zum Modellieren von geschwungenen, runden Körperformen.
2. Das Erstellen einer Methode, die morphologische Mannigfaltigkeiten als gerichtete Graphen speichert.
3. Das Beschreiben eines Verfahrens, welches diese Graphen nach geeigneten Bewegungsabläufen durchsucht.
4. Das Entwerfen eines modularen, 3D-druckbaren Steckkits, mit dessen Hilfe sich besagte Morphologien und Bewegungsabläufe schnell in Betrieb nehmen und evaluieren lassen.

Das folgende Kapitel führt den aktuellen Stand der Forschung ein. Es werden relevante Publikationen aus verschiedenen Teilgebieten aufgezeigt und deren Ergebnisse und Vorgehensweisen verglichen. Das Kapitel „Grundlagen“ illustriert die Themen, auf welchen die Arbeit aufbaut. Darunter befinden sich Roboterposen, dynamische Systeme, morphologische Mannigfaltigkeiten sowie ein kleiner Ausflug in die Graphentheorie. Das Kapitel „Konzept“ widmet sich dem entwickelten Gestaltungsprozess. Robotermorphologien, modelliert durch Kreisbögen, erzeugen eine morphologische Mannigfaltigkeit, welche nach geeigneten Bewegungsabläufen durchsucht wird. Am Ende führt dieses Kapitel das modulare Robotersteckkit namens LARA ein, wodurch neue Roboter schnell 3D-gedruckt, montiert und in Betrieb genommen werden können. Das fünfte Kapitel evaluiert den Prozess mit zwei Versuchsreihen: Die erste vergleicht verschiedene Bewegungsstrategien an derselben Morphologie und die zweite testet ein Verhalten auf vier verschiedenen Robotern. Abschließend fasst das Fazit die Ergebnisse zusammen und zeigt Ansatzpunkte für weiterführende Arbeiten auf.

2 Stand der Forschung

Eine verbreitete und mehrfach bewährte Methode zum Verbessern von Robotermorphologien bietet die *künstliche Evolution*. Es handelt sich um ein stochastisches Optimierungsverfahren, welches der biologischen Evolution nachempfunden ist. Generationen von Individuen mit zufällig gewürfelten Genen müssen sich gegen ihre Wettstreiter bewähren, um ihre Gene an die nächste Generation weiterzugeben. Sims bewies bereits 1994, welche Auswirkung das gleichzeitige Evolvieren von Morphologie und neuronalen Reglern – die Koevolution von Körper und Gehirn – hat[2]. Die simulierten Kreaturen entwickeln dadurch komplexe Verhaltensweisen, die genau auf ihre Körpereigenschaften zugeschnitten sind.

Bongard knüpft daran an, indem er die Entwicklung der Morphologie auf verschiedenen Zeitskalen erforscht[3]. Auf der einen Seite durchleben die Individuen eine ontogenetische Veränderung: Im Laufe ihres Lebens werden bestimmte morphologische Parameter kontinuierlich angepasst. Beispielsweise starten die Kreaturen mit einer beinlosen Aalform und über die Zeit wachsen ihnen Beine. Auf der anderen Seite erfahren sie auch eine phylogenetische Veränderung: Die Morphologie ändert sich von Generation zu Generation. Das bedeutet, dass die Nachkommen leicht unterschiedliche Körpereigenschaften wie ihre Elterngeneration vorweisen, mit denen die Individuen umgehen müssen. Verglichen mit der reinen Evolution des Gehirns, zeigen die Kreaturen mit kontinuierlich wachsenden Körpern robustere, intuitivere Bewegungen. Die aalförmigen Kreaturen lernen bereits in den frühen Lebensstadien ihren Körper schlangenartig fortzubewegen, anstatt sich nur auf die Beine zu verlassen.

Künstliche Evolution hat den Vorteil, dass durch große Rechenkapazitäten auch große Mengen an Parametern evolviert und viele Generationen mit geringem Zeitaufwand getestet werden können. Allerdings ist die evolvierte Lösung immer nur so gut, wie der verwendete Simulator. Letztendlich entdecken die Individuen jede realitätsfremde Lücke des Simulators und nutzen diese aus. Das bedeutet wiederum, dass sich die erzeugten Regler und Morphologien in der Realität ganz anders verhalten können. Dieses Phänomen ist als das *Simulation to Reality Gap*[4] bekannt. Brodbeck et al. führten

bereits Experimente durch, welche die künstliche Evolution vom Simulator in die reale Welt überführen[5]. Jedoch ziehen die begrenzten Zeit- und Materialressourcen der künstlichen Evolution einen Strich durch die Rechnung. Nicht nur können viel weniger Individuen und Parameter getestet werden, sondern auch der Raum der möglichen Genkombinationen wird wegen physikalischer Einschränkungen drastisch verkleinert. Die Evolution neigt dazu, in lokalen Maxima zu verharren und keine neuen Verhaltensweisen auszuprobieren.

Die Hürden der realen Welt sind der Robotik nur allzu bekannt. Um in einer komplexen, unvorhersehbaren Welt nicht zu scheitern, werden Roboter mit vielen Sensoren und motorischen Freiheitsgraden ausgestattet. Die sich ständig verbessernde Technologie spielt dieser Entwicklung in die Hände und ermöglicht es, Roboter mit preiswerten und hochwertigen Peripherien zu bauen. Doch erhöht jeder neue Sensor und jedes zusätzliche Gelenk die Dimension des Eingangssignals. Das System wird dadurch komplizierter und erfordert wiederum eine aufwendigere Datenverarbeitung. Sensomotorische Zusammenhänge, die sich dem dreidimensionalen Raum entziehen, sind für Menschen kaum noch nachzuvollziehen. Richard E. Bellman taufte dieses Problem den *Fluch der Dimensionalität*. Es besteht also, besonders in der Robotik, ein Bedarf für eine Methode namens *Dimensionsreduktion*. Die Aufgabe besteht darin, einen hochdimensionalen Datensatz auf einen Raum mit weniger Dimensionen abzubilden, sodass die markantesten Zusammenhänge bestmöglich herausgefiltert werden. Für die Robotik hat sich die *Slow Feature Analysis* bewährt[6]. Sie betrachtet die Datenpunkte aus einer zeitlichen Perspektive. Die entstehenden reduzierten Dimensionen bilden Eigenschaften ab, die sich zeitlich nur langsam verändern und somit von Sensorrauschen befreit sind. Die NASA erforschte mit dem humanoidem Weltraumroboter *Robonaut* welche Trajektorien bestimmte Bewegungen im Zustandsraum der reduzierten Dimensionen abfahren[7]. Durch das Wiederholen einer bestimmten Bewegung, wie zum Beispiel dem Greifen nach einem Objekt, bewegen sich die Sensordaten zumeist auf Mannigfaltigkeiten, mit nur wenig Dimensionen.

Eine Dimensionsreduktion ist eine aufwendige Rechenoperation. Allerdings liefern auch simplere Algorithmen sinnvolle und echtzeitaugliche Ergebnisse. Das *Attractor-Based-*

2 STAND DER FORSCHUNG

Behavior-Control (ABC)[8] ist ein exploratives Lernverfahren, mit dem ein Roboter mit beliebiger Morphologie seine hochdimensionale Mannigfaltigkeit autonom erforscht. Durch geschicktes Umschalten der Gelenkansteuerungen und Ausnutzen der Attraktorlandschaft, findet der Roboter interessante Schlüsselposen ganz von allein. Heuristiken können das ABC-Verfahren erweitern und bestimmte Erforschungsstrategien hervorruufen, wodurch die Mannigfaltigkeit schneller oder vorsichtiger erforscht wird[9].

Ein weiteres Feld, welches sich mit den Möglichkeiten von Morphologie beschäftigt, sind *selbstkonfigurierende, modulare* Roboter. Das Ziel ist die Entwicklung eines robotischen Systems, das aus einzelnen Modulen besteht. Die Besonderheit dabei ist, dass eine große Anzahl von Modulen derselben Sorte zur Verfügung steht und sich zu beliebigen Konfigurationen zusammensetzen lässt. Das System kann dadurch ein breites Spektrum an Morphologien verkörpern. Passend zur Umgebung oder Aufgabe, kann sich der Roboter umkonfigurieren, selbst reparieren oder sogar reproduzieren. Neben der Vielseitigkeit der Verhaltensmöglichkeiten verspricht die modulare Robotik Robustheit durch Redundanz von einzelnen Modulen und geringe Herstellungskosten durch eine mögliche Massenproduktion[10].

Einer der ersten modularen Roboter ist PolyBot von Yim et al. aus dem Jahr 2000[11]. Jedes der würfelförmigen Module enthält einen motorischen Freiheitsgrad und besitzt zwei Flächen mit einem standardisierten elektrisch-mechanischem Verbindungssystem. PolyBot kann bereits von einer schlangenförmigen, Sinus-angesteuerten Gangart in eine ringförmige Panzerketten-Gangart übergehen und damit die Fortbewegung effizient auf seine Umwelt anpassen.

Es handelt sich dabei um *deterministische* Rekonfiguration. Auf der Mikroskala existiert noch ein anderer Ansatz: stochastische Rekonfiguration. Diese beruht darauf, dass viele Module frei in einer Flüssigkeit schwimmen und sich über einen Andockmechanismus an eine Basisstruktur anfügen. In [12] zum Beispiel wurden würfelförmige Module in einem Öltank versenkt, die sich entweder durch Elektromagnete oder durch Saugkraft miteinander verbinden und beliebige vorgegebene Strukturen annehmen können.

Im Gegensatz zu standardisierten Würfelmodulen, die eine hohe Vielseitigkeit versprechen, gibt es auch humanoide, modulare Roboter. Der Fokus liegt dabei weniger auf

der selbstständigen Rekonfiguration, als auf der Geschlossenheit und Selbstständigkeit einzelner Module. Im Forschungslabor Neurorobotik in Berlin entwickelten Hild et al. den humanoiden Roboter *Myon*[13]. Dieser lässt sich dank verteilter Spannungsversorgungen und Recheneinheiten während der Inbetriebnahme auseinandernehmen und neu zusammensetzen. Diese Dezentralisierung erleichtert die Wartung von beschädigten Teilen, ermöglicht gleichzeitiges Forschen an verschiedenen Körperteilen und bleibt anpassungsfähig für zusätzliche Körperteile oder Modulverbesserungen.

Auch um Kreativität und Begeisterung in Kindern hervorzurufen, haben sich modulare Roboterkits bereits bewiesen. Kommerzielle Systeme wie *LEGO-Technik*, *Cubelets* oder *Robo Wunderkind* sind beliebte Robotersysteme, um besonders junge Kinder an Themen wie Konstruktion oder Programmierung heranzuführen. Eines der wegweisenden Robotikkits ist *Topobo*[14]. Es besteht aus einer Handvoll passiver Module und einem Motormodul mit *kinetischem Gedächtnis*. Auf Knopfdruck nimmt es manuell erzeugte Bewegungen auf und führt diese dann periodisch aus. Experimente an Schulen zeigten, dass Kinder iterative Gestaltungs- und Programmierprozesse entwickeln und sich von Erfahrungen mit ihrem eigenen Körper inspirieren lassen.

3 Grundlagen

Dieses Kapitel führt die wesentlichen theoretischen Prinzipien ein, von denen der Rest der Arbeit Gebrauch macht. Roboter mit motorisierten Gelenken können mit ihrem Körper eine *Pose* einnehmen. Diese Posen unterliegen der Schwerkraft, weshalb sie aus der Perspektive der *dynamischen Systeme* unter die Lupe genommen werden können. Im Zustandsraum eines solchen Systems bilden die stabilen Posen eine Mannigfaltigkeit. Diese kann mithilfe der Graphentheorie als eine Datenstruktur dargestellt werden und dem Roboter als ein Körpermodell dienen.

3.1 Roboterposen

Als Pose bezeichnet man die Körperhaltung von Lebewesen oder Robotern. Sie hängt von der Konfiguration der Gelenke und der Orientierung des Körpers zur Umwelt ab. Bei Menschen würde man Stehen, Sitzen oder Liegen als Posen bezeichnen. Beim Stehen sind die Kniegelenke ausgestreckt, beim Sitzen jedoch angewinkelt. Stehen und Liegen unterscheiden sich durch ihre Gelenkkonfiguration hingegen kaum. Stattdessen befinden sich beim Stehen die Beine und der Torso rechtwinklig zum Boden, während sie beim Liegen parallel zu ihm verlaufen.

Abbildung 1 illustriert einen Beispielroboter namens *Rectbot*. Er besteht aus zwei rechteckigen Körperteilen, die durch ein motorisiertes Gelenk verbunden sind, welches in der Abbildung als ein schwarzer Kreis eingezeichnet ist. Die weißen Punkte markieren die Massenschwerpunkte der einzelnen Körperteile. Die Masse eines Körperteils ist pro-

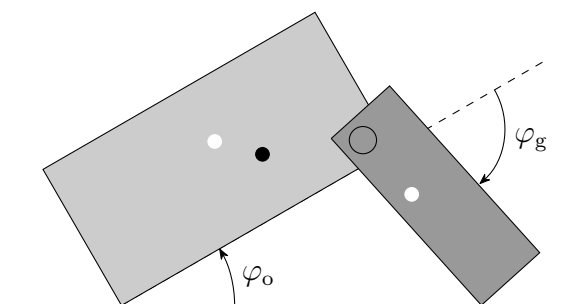


Abbildung 1: Rectbot mit der Pose $P = (30^\circ, -78^\circ)$

portional zu seinem Flächeninhalt, weshalb sich der gemeinsame Massenschwerpunkt – der schwarze Punkt in der Abbildung – näher am großen Körperteil befindet. Der schwarze Strich unter dem Roboter stellt den Boden dar, auf dem Rectbot mit zwei Eckpunkten aufliegt.

Der der Orientierungswinkel φ_o und der Gelenkwinkel φ_g von Rectbot beschreiben eindeutig seine Pose:

$$P = (\varphi_o, \varphi_g) \in \mathbb{R}^2$$

Die Menge aller möglichen Posen, die Rectbot einnehmen kann, ist in Abbildung 2 zu sehen. Die beiden Winkel sind 2π -periodisch, das heißt, wenn sich einer der Winkel über den Rand der dargestellten Fläche bei $\pm\pi$ hinaus bewegt, taucht er auf der anderen Seite wieder auf. Alternativ können Gelenke Anschläge haben, die die Periodizität unterbrechen. In dem Fall wären die Achsen auf ein bestimmtes Intervall beschränkt. Die möglichen Posen können weiterhin durch *Selbstkollision* eingeschränkt werden. Unter Selbstkollision versteht man die Einschränkung einer Bewegung durch den eigenen Körper. Selbstkollision verhindert beispielsweise die Bewegung des einen Beins durch das andere. Für Rectbot nehmen wir weder Anschläge noch Selbstkollision an, wodurch der gesamte $\mathbb{R}^2$ zur Verfügung steht.

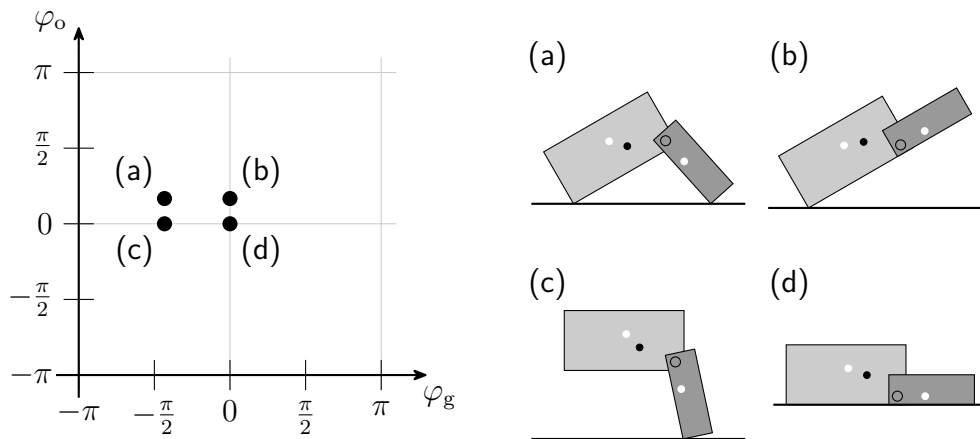


Abbildung 2: Die möglichen Posen von Rectbot, als zweidimensionale Punktmenge dargestellt. Die zwei Beispielposen (a) und (d) sind, anders als (b) und (c), stabil, da sie nicht umfallen.

3 GRUNDLAGEN

Dabei fällt auf, dass sich die Posen in *stabil* und *instabil* kategorisieren lassen. Stabile Posen zeichnen sich dadurch aus, dass sie sich von allein nicht verändern, während instabile Posen eben das tun. Sie fallen durch den Einfluss von Schwerkraft um. Wenn eine instabile Pose umfällt, verändert sich ihr Orientierungswinkel φ_o , und zwar so lange, bis sich eine stabile Pose ergibt. Durch eine gemäßigte Bewegung des Gelenks würde der Roboter sich ausschließlich über stabile Posen von (a) nach (d) bewegen. Instabile Posen wie (b) oder (c) sind dadurch unerreichbar. Dieser Einfluss der Schwerkraft auf eine Pose lässt sich durch ein dynamisches System modellieren.

3.2 Dynamische Systeme

Ein dynamisches System ist eine Funktion f , die für einen Zustand x die zeitliche Ableitung vorgibt:

$$x'(t) = f(x(t)) \quad \text{mit} \quad x(t) \in \mathbb{R}^n$$

Wobei $x(t)$ einen n -dimensionalen Punkt repräsentiert. Dadurch lässt sich für beliebige Startzustände $x(0)$ das Verhalten über die Zeit t analysieren. Die Zustände, die ein dynamisches System für $t \rightarrow \infty$ annimmt, werden als *Attraktoren* bezeichnet.

Um das Beispiel aus dem vorherigen Abschnitt aufzugreifen: Die Pose eines Roboters kann unter dem Einfluss der Schwerkraft als Zustand eines dynamischen Systems betrachtet werden. Es könnte wie folgt modelliert sein:

$$\varphi_o'(t) = \begin{cases} +\omega_o, & c_x < p_x \\ -\omega_o, & c_x > p_x \\ 0, & c_x = p_x \end{cases} \quad (1)$$

Wobei $c = (c_x, c_y)$ der Massenschwerpunkt des Roboters und $p = (p_x, p_y)$ der Auflagepunkt ist. Der Auflagepunkt ist der aktuell tiefste Punkt des Roboters, also an dem der Roboter den Boden berührt. Ein Vergleich der x-Koordinaten genügt, um die Fallrichtung zu bestimmen. Befindet sich c weiter links als p , also sobald $c_x < p_x$ gilt, kippt der Roboter nach links um. Das Umfallen nach links zeigt sich hier als eine positive Winkelgeschwindigkeit φ_o' . Der Betrag ist durch die Konstante ω_o modelliert.

Analog gilt für $c_x > p_x$ eine negative Winkelgeschwindigkeit. Der dritte Fall bei $c_x = p_x$ beschreibt die Posen, die auf einer Ecke balancieren, ohne umzufallen. Dieser Zustand ist allerdings nur theoretisch stabil, da die kleinste Störung c verschiebt und die Pose zum Umfallen bewegt.

Es sei an dieser Stelle klargestellt, dass es sich bei Gleichung 1 um eine Vereinfachung der realen Welt handelt. Die Schwerkraft beeinflusst eigentlich die Winkelbeschleunigung φ''_o . Allerdings genügt für langsame Bewegungen das einfache Modell, um die Eigenschaften der Pose zu analysieren.

Abbildung 3 zeigt den *Phasenraum* des dynamischen Systems, wenn Rectbot sein Gelenkwinkel auf -78° festgestellt hat. Die Fallrichtung $\text{sign}(\varphi'_o)$ wird durch Pfeile auf der zirkulären Achse repräsentiert. Wandert der Zustand entlang der Pfeile, gelangt das System irgendwann in einen Attraktor, nämlich in einen der vier *stabilen Fixpunkte*, welche sich in der Abbildung bei den schwarzen Punkten befinden. Die entsprechenden Posen zeichnen sich dadurch aus, dass sie nicht von allein umfallen. Alle Zustände, die zum gleichen Attraktor streben bezeichnet man als *Bassin*, das französische Wort für Becken. Am Rande dieser Bassins befinden sich *instabile Fixpunkte*, hier als weiße Punkte dargestellt. Ein instabiler Fixpunkt entspricht einer Pose, für welche $c_x = p_x$ gilt. Hierbei handelt es sich um den dritten Fall aus Gleichung 1, bei welcher der Roboter auf einer seiner Ecken balanciert wie beispielsweise bei $\varphi_o = -42^\circ$ in Abbildung 3. Bei der kleinsten Störung fällt der Zustand in eines der benachbarten Bassins und nähert sich einem stabilen Fixpunkt.

Der Gelenkwinkel φ_g , also die relative Drehung zwischen den beiden Körperteilen, fungiert hier als ein Parameter, mit dem das System beeinflusst und die Attraktoren verschoben werden können. Durch das Variieren von Parametern kann es in dynamischen Systemen allerdings zu einer *Bifurkation* kommen. Das heißt, dass sich die Anzahl oder Art der Attraktoren verändert. In unserem Fall bedeutet eine Bifurkation, dass durch die Gelenkbewegung eine stabile Pose plötzlich instabil wird oder umgekehrt. Analysiert man die Veränderung von Rectbots Phasenraum durch das Variieren von φ_g , entsteht das Bifurkationsdiagramm aus Abbildung 4 auf Seite 14. Die schwarzen Linien stellen stabile Fixpunkte dar und wie sie sich durch φ_g beeinflussen lassen. Bei den rot- und

3 GRUNDLAGEN

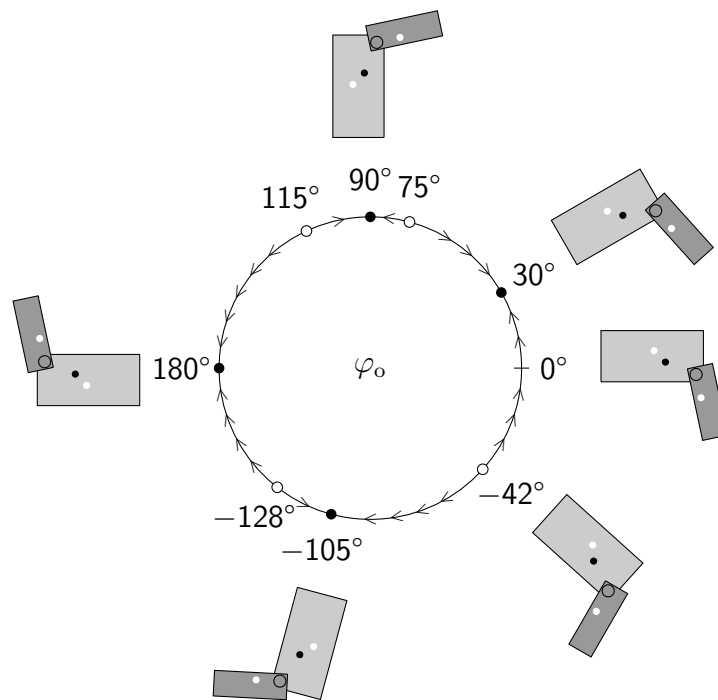


Abbildung 3: Hält Rectbot sein Gelenk bei $\varphi_g = -78^\circ$ konstant, entsteht dieser Phasenraum. Die Pfeile auf der Achse zeigen die Fallrichtung an. Die Posen, bei denen die Pfeile aufeinandertreffen, sind stabile Fixpunkte. Analog finden sich die instabilen Fixpunkte dort, wo sich die Pfeile voneinander entfernen.

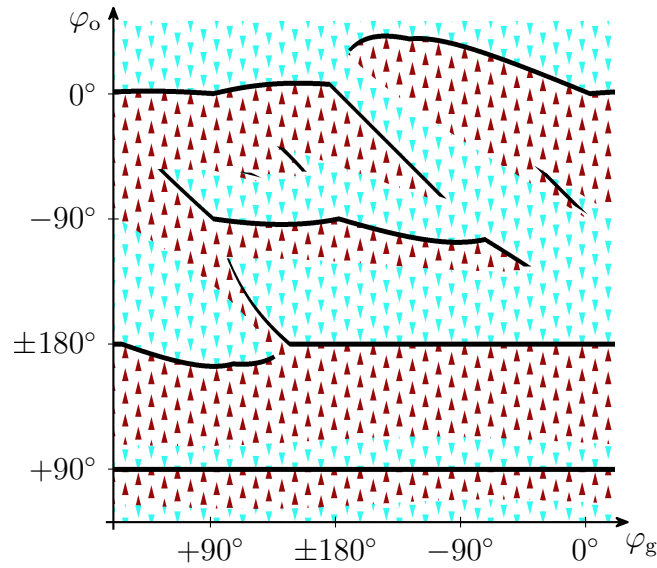


Abbildung 4: Rectbots Bifurkationsdiagramm

türkisfarbenen Pfeilmustern handelt es sich um instabile Posen, die in Richtung der Pfeile umfallen, bis sie auf eine schwarze Linie treffen. Mit einer Ansteuerung seines motorisierten Gelenks kann sich der Roboter entlang dieser Linien bewegen. Bei den Stellen, an denen eine der Linien aufhört, handelt es sich um eine Bifurkation. Hier wird die Pose durch die Gelenkbewegung plötzlich instabil und fällt um. Die Zusammenhänge zwischen Abbildung 4 und den stabilen Posen von Rectbot lassen sich am besten interaktiv im Shadertoy *Manifolds of Rectbot*¹ nachvollziehen.

3.3 Morphologische Mannigfaltigkeiten

Gelten Gelenk- und Orientierungswinkeln eines Roboters als Zustand eines dynamischen Systems, bilden die Attraktoren eine Mannigfaltigkeit:

$$\mathcal{M} = \{(\varphi_o, \varphi_g) \in \mathbb{R}^2 \mid F(\varphi_o, \varphi_g) = 0\}$$

Die charakteristische Funktion F ist dann gleich 0, sobald die Winkel eine stabile Pose hervorrufen. Die in Abbildung 4 schwarz gezeichneten Linien repräsentieren die Winkelpaare (φ_o, φ_g) , die zu Rectbots morphologischer Mannigfaltigkeit gehören.

¹<https://www.shadertoy.com/view/7s2XDy>

3.3.1 Mehr Dimensionen

Die Mannigfaltigkeit lässt sich auch für Roboter in einer dreidimensionalen Welt wie der unseren entschlüsseln. Anders als in dem bislang beschriebenen zweidimensionalen Flachland von Rectbot, gibt es dort statt einem Orientierungswinkel allerdings gleich drei, beispielsweise die eulerschen Winkel. Doch auch dafür lässt sich nach dem Vorbild von Gleichung 1 ein dynamisches System zur Bestimmung der stabilen Posen beschreiben: Die Projektion des Massenschwerpunktes auf die Bodenfläche ergibt einen zweidimensionalen Punkt. Befindet sich dieser innerhalb des konvexen Polygons der Auflagepunkte, in der Regel einem Dreieck, ist die Pose stabil, andernfalls kippt der Roboter über eine der Kanten des Polygons, bis ein neuer Punkt den Boden berührt. Die Konzepte gelten auch für Roboter mit mehr als einem Gelenk. Jeder zusätzliche Freiheitsgrad entspricht einem weiteren Parameter des dynamischen Systems, mit dem die Attraktoren beeinflusst und Bifurkationen erzeugt werden können. Für ein weiteres Gelenk lässt sich die Mannigfaltigkeit noch visualisieren. Abbildung 5 führt einen neuen Roboter namens *Tablebot* ein. Dieser hat nun zwei Gelenke: am linken (φ_l) und am rechten Bein (φ_r). Zusammen mit dem Orientierungswinkel ist der Zustandsraum also dreidimensional. Die Mannigfaltigkeit etabliert sich darin als gewölbte Flächen wie in Abbildung 6. Die drei verschiedenen Perspektiven sollen die Dreidimensionalität verdeutlichen. Jeder Punkt darin entspricht einer stabilen Pose. Die Farbe des Punktes symbolisiert seine Zugehörigkeit zu einer *Untermannigfaltigkeit* $\mathcal{U} \subset \mathcal{M}$. Alle Posen einer Untermannigfaltigkeit, also Punkte mit derselben Farbe, lassen sich untereinander ansteuern ohne umzufallen. Schwarze Ränder stehen für physikalische Einschränkungen

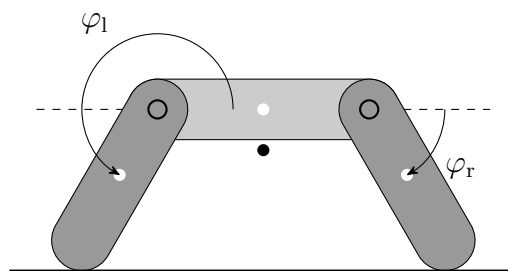


Abbildung 5: Tablebot mit der Pose $P = (0, 4\pi/3, -\pi/3)$

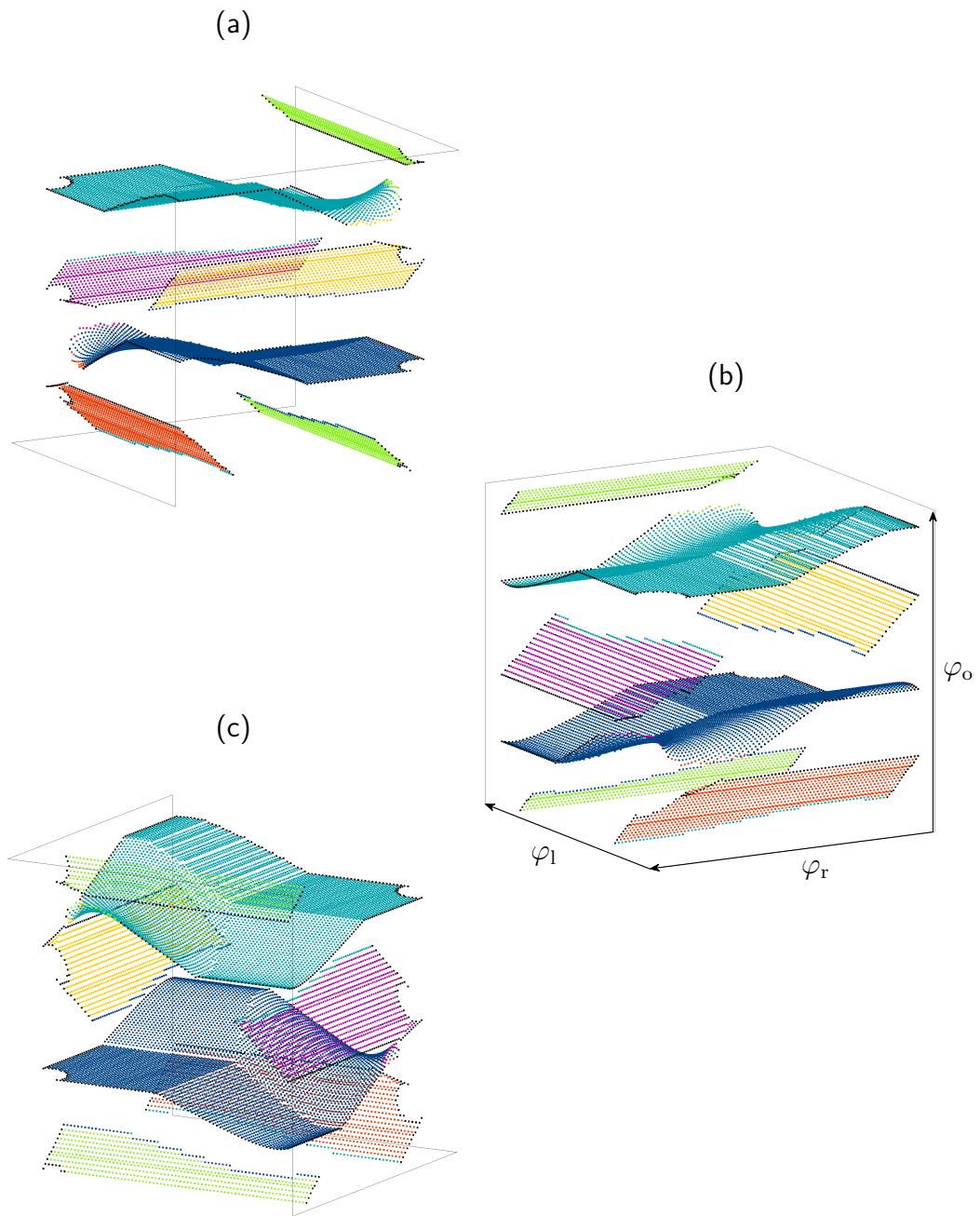


Abbildung 6: Tablebots dreidimensionale morphologische Mannigfaltigkeit aus drei verschiedenen Ansichten. (a) ist 30° um die senkrechte Achse gedreht, (b) um 120° und (c) um 240° . Jede Achse hat eine Auflösung von $n = 80$.

3 GRUNDLAGEN

wie etwa ein Gelenkanstoß oder eine Selbstkollision. Farbige Ränder sind Bifurkationen und bedeuten eine Fallbewegung. Bewegt sich der Roboter über einen solchen Rand hinaus, fällt sein Zustand in die Untermannigfaltigkeit, welche die Farbe des Randpunktes trägt.

Bei Tablebot sind φ_l und φ_r auf ein Intervall beschränkt. Der Orientierungswinkel hingegen hat keine Einschränkung. Bewegt sich der Zustand oben aus dem abgebildeten Diagramm heraus, würde er unten wieder auftauchen. Die obere und untere grüne Fläche gehören deswegen zur selben Untermannigfaltigkeit.

Um der wachsenden Anzahl der Posen durch zusätzliche Dimensionen entgegenzuwirken, muss nicht unbedingt eine aufwendige Dimensionsreduktion durchgeführt werden. Oftmals haben einige Gelenke bei bestimmten Posen keinen Einfluss auf die Orientierung. Auf den flachen Ebenen in Abbildung 6 (rot, grün, lila und gelb), ist die Orientierung in Richtung einer Dimension unveränderlich. Dort hält der Roboter eines seiner Beine in die Luft und kann dieses ohne Bedenken bewegen. Die türkisfarbene und die rote Untermannigfaltigkeit haben sogar einen Bereich, der in beiden Gelenkdimensionen unveränderlich ist. Die Auflösung von $\mathcal{M}$ kann an solchen Stellen verringert oder die einzelnen Punkte ganz durch Ebenengleichungen ersetzt werden. [15] bewältigt das analytische Beschreiben einzelner Untermannigfaltigkeiten mit Quadriken.

3.3.2 Interpretation der Mannigfaltigkeit

Diese Mannigfaltigkeiten sind implizite Körpermodelle des Roboters. Sie mögen auf den ersten Blick kryptisch erscheinen, doch es lassen sich direkte Zusammenhänge zur Morphologie des Roboters erkennen.

Beispielsweise spiegelt sich die Symmetrie von Tablebot in seiner Mannigfaltigkeit wider. Dreht man ihn um 180° und stellt ihn auf den Kopf, hat man exakt denselben Roboter vor sich. In der Mannigfaltigkeit entsteht dadurch eine punktsymmetrische Spiegelung. Die zwei großen Untermannigfaltigkeiten türkis und blau haben genau dieselbe Form, nur gespiegelt. Innerhalb dieser Untermannigfaltigkeit lässt sich eine weitere Symmetrie erkennen: Der Einfluss von φ_l auf φ_o ist vom Betrag derselbe wie der von φ_r , allerdings in die andere Richtung. Wandert man ausgehend von der waa-

gerechten türkisfarbenen Fläche entlang φ_r , beginnt die Fläche sich nach oben zu krümmen, während bei φ_l die Krümmung abwärts erfolgt. Das ist auf die Gleichheit der zwei Beine zurückzuführen. Erkennt der autonome Roboter diese Symmetrien, kann er Verhaltensweisen, die auf der einen Seite funktionieren, direkt auf die andere Seite übertragen.

3.4 Graphentheorie

Die Graphentheorie ist das Teilgebiet der Mathematik, welches sich mit Netzwerken beschäftigt. Netzwerke bilden die grundlegende Datenstruktur für vielerlei alltägliche Dinge wie öffentliche Verkehrsnetze, Familienstammbäume oder chemische Moleküle. Sie bestehen aus einzelnen Knoten v , die Teil einer Knotenliste V sind: $v \in V$. In den eben genannten Beispielen sind die Knoten stellvertretend für Haltestellen, Familienmitglieder oder Atome. Diese Knoten werden nun beliebig untereinander mit *Kanten* e verbunden, die ihrerseits wiederum Teil einer Kantenliste E sind:

$$e \in E \subset \{(v, u) \mid v, u \in V\}$$

Kanten repräsentieren Beziehungen zwischen zwei Knoten. Zwei Haltestellen sind mit einer Buslinie verbunden. Im Stammbaum kennzeichnet die Kante eine Verwandtschaft. In Molekülen werden die Atome durch chemische Bindungen zusammengehalten. Die Definition eines Graphen $\mathcal{G}$ ist also:

$$\mathcal{G} = (V, E)$$

Es gibt viele verschiedene Arten von Graphen, die sich alle durch bestimmte Regeln unterscheiden. Abbildung 7 zeigt eine für diese Arbeit relevante Auswahl von Graphen. Einer davon ist ein *gerichteter* Graph, dessen Kanten mit einer Richtung behaftet sind. In Abbildung 7 sind die Richtungen durch einen Pfeil verdeutlicht. Für sie gilt folgende Regel:

$$(v, u) = v \rightarrow u \quad \neq \quad (u, v) = u \rightarrow v$$

Die Reihenfolge, in welcher die Knoten einer Kante gespeichert sind, macht einen Unterschied. Eine Buslinie fährt zumeist in beide Richtungen, aber eine Mutter-Tochter

3 GRUNDLAGEN

Beziehung gilt nur in eine Richtung. Um die Beziehung von zwei Knoten zu verdeutlichen, erhält der Ausgangsknoten v einer gerichteten Kante $v \rightarrow u$ die Bezeichnung *Elter* und analog dazu heißt der Zielknoten u auch *Kind*.

Eine Bewegung über mehrere zusammenhängende Kanten wird *Pfad* genannt. Gelangt man dadurch zu demselben Knoten, bei welchem der Pfad begonnen wurde, handelt es sich um einen *Kreis*. In manchen Netzwerkstrukturen sind Kreise verboten. Ein Beispiel dafür wäre der Stammbaum. Da sich bei solchen Graphen die Knoten wie Äste von einem Stamm ausbreiten, werden sie als *Bäume* bezeichnet. Auch Bäume können gerichtete Kanten haben. Sind diese so ausgelegt, dass alle Knoten von einem besonderen Knoten ausgehen, wird dieser als *Wurzel* bezeichnet. In der Abbildung ist die Wurzel mit einem w gekennzeichnet. Für solche *gewurzelten Bäume* ist jeder Knoten indirekt von der Wurzel aus erreichbar.

3.4.1 Mannigfaltigkeit als Graph

Mit den stabilen Posen als Knoten kann die morphologische Mannigfaltigkeit ebenfalls durch einen Graphen dargestellt werden. Die Posen, die durch Motoransteuerungen direkt erreichbar sind, werden mit Kanten verbunden. Da diese Ansteuerungen nicht immer für beide Richtungen gelten, nämlich wenn der Rand einer Untermannigfaltigkeit überschritten wurde, ist ein gerichteter Graph die passende Darstellungsform.

Die Graphendarstellung eignet sich besonders, da sie sich dynamisch erweitern lässt.

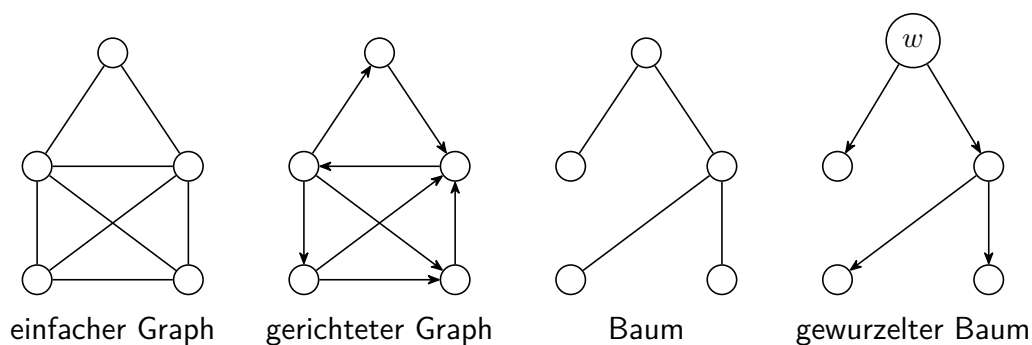


Abbildung 7: Verschiedene Arten von Graphen

Dadurch können autonome Roboter, die kein anfängliches Wissen über ihre Körperform haben, ihre morphologische Mannigfaltigkeit auch explorativ in Echtzeit erstellen und sich ein eigenes Körpermodell aufbauen, wie in [8] und [9] gezeigt wird.

4 Konzept

Abbildung 8 schlägt einen Gestaltungsprozess für autonome Roboter vor, der die Eigenschaften der Morphologie, konkreter morphologischer Mannigfaltigkeiten, berücksichtigt und als Grundlage für verhaltenssteuernde Algorithmen nutzt. Es sollen möglichst unkompliziert Robotertermorphologien entworfen, ihre Mannigfaltigkeiten nach Verhaltensweisen analysiert und beides an einem verkörpertem Roboter getestet werden können. Erkenntnisse aus der Evaluation in der realen Welt fließen dann in einem iterativen Vorgehen in den nächsten Entwurf mit ein.

1. *Entwurf*: Die Morphologien von Robotern werden als gewurzelte Baumgraphen gespeichert. *Kreisbögen* dienen als geometrisches Werkzeug, um runde, realistische Körperkonturen zu erzeugen.
2. *Analyse*: Von diesen Robotern wird als Nächstes die morphologische Mannigfaltigkeit erstellt und als gerichteter Graph gespeichert. Dieser kann aufgabenabhängig nach günstigen Bewegungsabläufen durchsucht werden.
3. *Einsatz*: Die Morphologie und Bewegungsabläufe können dann real evaluiert werden. Das modulare Robotersteckkit LARA ermöglicht eine flexible und schnelle Inbetriebnahme neuer Morphologien.

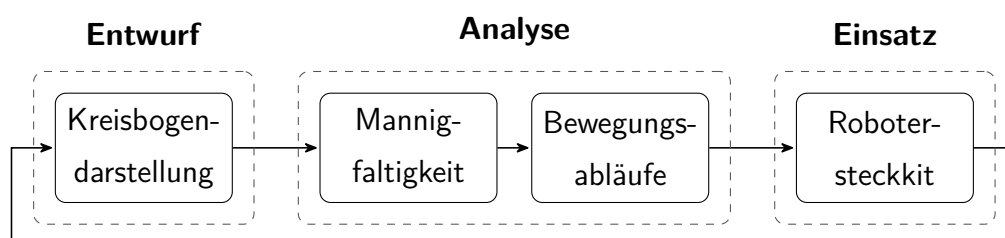


Abbildung 8: Schematische Übersicht eines Gestaltungsprozesses für autonome Roboter. Die gewonnenen Ergebnisse aus Analyse und Einsatz können in einem iterativen Vorgehen zur Weiterentwicklung der Morphologie verwendet werden.

4.1 Darstellung eines Roboters

Um Robotermorphologien zu entwerfen, muss eine Datenstruktur definiert werden, die ein Modell des Roboters abbilden kann. Dieses Modell sollte alle nötigen Informationen enthalten, um einen Roboter real nachbauen zu können und die Erwartungen aus der berechneten Mannigfaltigkeit den physikalischen Eigenschaften entsprechen zu lassen. Das Motto lautet dabei: So simpel wie möglich, aber so komplex wie nötig.

4.1.1 Roboter-Datenstruktur

Gewurzelte Bäume eignen sich, um die Morphologie eines zweidimensionalen Roboters als eine Datenstruktur namens $\mathcal{R}$ abzuspeichern. Die Knoten $\tau_i \in V$ von $\mathcal{R}$ sind die einzelnen formfesten Körperteile. Eine Kante des Graphen gibt an, welche zwei Körperteile τ_e und τ_k durch ein rotatorisches Gelenk aneinanderhängen. Der Graph wird also folgendermaßen definiert:

$$\begin{aligned} V &= \{\tau_1, \tau_2, \dots, \tau_n\} \\ E &\subset \{\tau_e \rightarrow \tau_k \mid \tau_e, \tau_k \in V\} \\ \mathcal{R} &= (V, E) \end{aligned}$$

Die Struktur des gewurzelten Graphen schreibt vor, dass erstens jeder Körperteil τ_k genau einen Elternteil τ_e besitzen muss und dass zweitens der Wurzelkörperteil $\mathcal{W} = \tau_1$ am Anfang des Stammbaumes sitzt und damit als einziger Teil keinen Elter hat.

Zu einem Körperteil gehört eine zweidimensionale geometrische Form $\mathcal{F}$. Diese wird durch die Kreisbogendarstellung in einem lokalen zweidimensionalen Koordinatensystem beschrieben. Der nächste Abschnitt geht auf die Kreisbogendarstellung genauer ein. Zusätzlich zur Geometrie hat aber auch die Masse m einen großen Einfluss auf das physikalische Verhalten des Körperteils. Sie kann durch einen einzigen Punkt, den Massenschwerpunkt c , modelliert werden, an dem äußere Kräfte wie Schwerkraft angreifen. In Roboterabbildungen sind die Schwerpunkte individueller Körperteile immer als weiße Punkte eingezeichnet.

Die Kante $\tau_e \rightarrow \tau_k$ beinhaltet alle nötigen Informationen über die Gelenkverbindung zwischen dem Elternteil τ_e und dem Kindteil τ_k . Die Lage des Gelenks $g = (g_x, g_y)$

4 KONZEPT

gibt die Position der Drehachse im lokalen Koordinatensystem von τ_e an, um die τ_k samt Masse und Form gedreht wird. Optional kann ein Gelenk noch Winkelanschläge $\alpha_{\min}$ und $\alpha_{\max}$ festlegen, über die sich der Gelenkwinkel nicht hinausbewegen kann.

4.1.2 Kreisbögen zum Modellieren von Körperformen

Ein Kreisbogen B ist eine kontinuierliche Teilmenge einer zweidimensionalen Kreislinie K mit dem Radius r um den Mittelpunkt $m = (m_x, m_y)$:

$$B \subset K = \{(x, y) \in \mathbb{R}^2 \mid r^2 = (x - m_x)^2 + (y - m_y)^2\}$$

Während die Kreislinie eine ganze Umdrehung um den Mittelpunkt macht, befindet sich der Kreisbogen zwischen einem Start- und Endpunkt, wie in Abbildung 9 zu sehen. Die gestrichelten Linien repräsentieren K und die durchgezogenen Abschnitte darauf sind B . Ein Kreisbogen lässt sich über fünf Parameter genau definieren: Startposition $p = (p_x, p_y)$, Startrichtung bzw. -winkel φ , Krümmung κ und Bogenlänge l . Die Krümmung κ ist gleich dem Kehrwert des Radius r :

$$|\kappa| = \frac{1}{r}$$

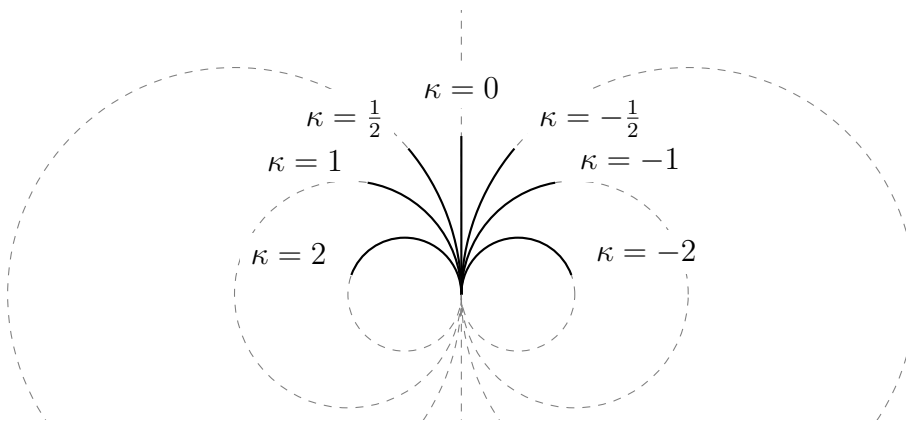


Abbildung 9: Einfluss der Krümmung κ auf Kreisbögen mit gleichbleibender Startposition, Startwinkel und Länge. Die korrespondierenden Kreise sind gestrichelt eingezeichnet. Bögen mit $\kappa = 0$ werden zu geraden Strecken.

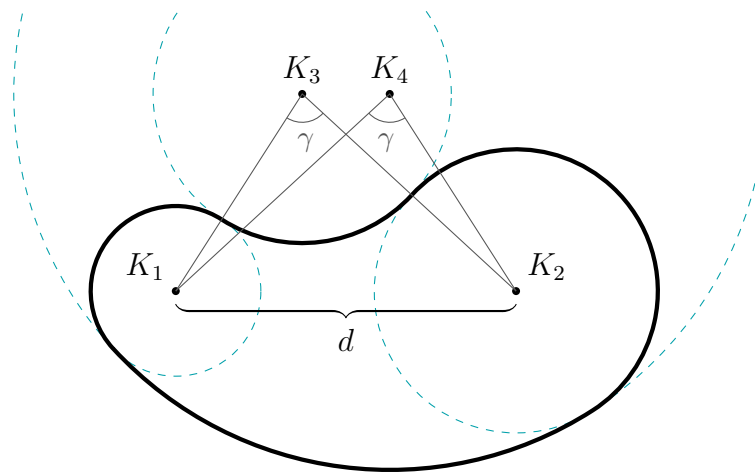


Abbildung 10: Die Kreisbogenform $\text{CASHEW}(6, \frac{3}{2}, \frac{5}{2}, 80^\circ)$ besteht aus vier Kreisbögen und kann eindeutig durch vier Parameter d , r_1 , r_2 und γ definiert werden.

Während der Radius immer positiv ist, schreibt das Vorzeichen $\text{sign}(\kappa)$ die Krümmungsrichtung vor. Positiv gekrümmte Bögen biegen nach links und negativ gekrümmte nach rechts. Krümmungen mit kleinen Beträgen erzeugen große Kreise und umgekehrt. Nähert sich die Krümmung der 0, wächst der Radius ins Unendliche, und aus der gekrümmten Kreislinie wird eine ungekrümmte Gerade. Abschnitte auf einer Geraden sind eigentlich keine Bögen mehr, sondern *Strecken*. In Abbildung 9 wird der flüssige Übergang von positiver Krümmung über gerade Strecken bis hin zu negativer Krümmung verdeutlicht, weshalb sich das Einbeziehen von Strecken als ungekrümmte Bögen $B_{\kappa \neq 0}$ für die Kreisbogendarstellung anbietet.

Ein tangenciales Aneinanderreihen von mehreren Kreisbögen zu einer geschlossenen Form ergibt eine Kreisbogenform $\mathcal{F} = \{B_1, B_2, \dots, B_n\}$. Abbildung 10 zeigt die sogenannte CASHEW-Form, bestehend aus nur vier Kreisbögen. Mit den richtigen Parametern ähnelt diese Form einem Cashewkern, woher auch ihr Name stammt. Die Körperteile von Tablebot sind beispielsweise aus solchen Formen modelliert. Vorteilhaft an der Kreisbogendarstellung ist, dass sich schon mit wenigen Bögen sinnvolle parametrisierte Formen erstellen lassen. Die CASHEW-Form ist über folgende vier Parameter einzustellen: Die Radien der Kreise K_1 und K_2 , der Abstand zwischen deren

4 KONZEPT

Mittelpunkten d und die Krümmung der zwei Verbindungskreise K_3 und K_4 , welche durch den Winkel γ einzustellen ist. Eine Form lässt sich demnach wie folgt konstruieren:

$$\mathcal{F} = \text{CASHEW}(d, r_1, r_2, \gamma)$$

Eine detailliertere mathematische Beschreibung der Kreisbögen und die Konstruktion der CASHEW-Form befindet sich im Anhang.

4.1.3 Kreisbogenformen im Vergleich mit Polygonen

Ein *Polygon* ist, ähnlich wie eine Kreisbogenform, eine geschlossene zweidimensionale geometrische Figur. Polygone werden aber durch eine gewisse Anzahl an Punkten definiert, die durch einen Streckenzug verbunden werden. Körper aus Polygonen haben deshalb immer spitze Ecken und flache Kanten. Derart idealisierte Formen finden sich in der Realität nur selten, besonders bei natürlichen Lebewesen. Ecken verschleifen schnell und stumpfen ab. Flache Kanten absorbieren beim Aufprall die Fallgeschwindigkeit, was einen Roboter beschädigen kann und einen potenziellen Schwungaufbau verhindert. Runde Konturen und tangentielle Verknüpfungen überwinden diese Einschränkungen. Roboter aus Kreisbögen wirken geschmeidiger und die Formen ähneln eher Körperteilen, die auch in der Natur anzutreffen sind. Bei einigen Fertigungsverfahren wie dem bei dieser Arbeit verwendeten 3D-Druck muss sowieso mit einem minimalen Eckradius gerechnet werden, welcher sich durch Kreisbögen problemlos einbauen lässt.

Zwar lassen sich runde Konturen durch hoch aufgelöste Polygone approximieren, jedoch wird durch eine große Anzahl Eckpunkte der Rechenaufwand erhöht. Außerdem weisen die Mannigfaltigkeiten auch bei hohen Auflösungen Unstetigkeiten auf. Abbildung 11 vergleicht die Mannigfaltigkeit einer Kreisbogendarstellung mit der einer Polygondarstellung. Ähnlich wie Rectbot besteht der hierfür verwendete Roboter *Bananabot* in (a) aus zwei Körperteilen. Für die Kreisbogendarstellung in (b) ist jeder Körperteil mit fünf Bögen modelliert (vgl. mit Shadertoy *Bananabot as Circular Arcs*²). Die Polygondarstellung in (c) hingegen approximiert die Körperteile mit je dreißig Eckpunkten (vgl.

²<https://www.shadertoy.com/view/flSGzc>

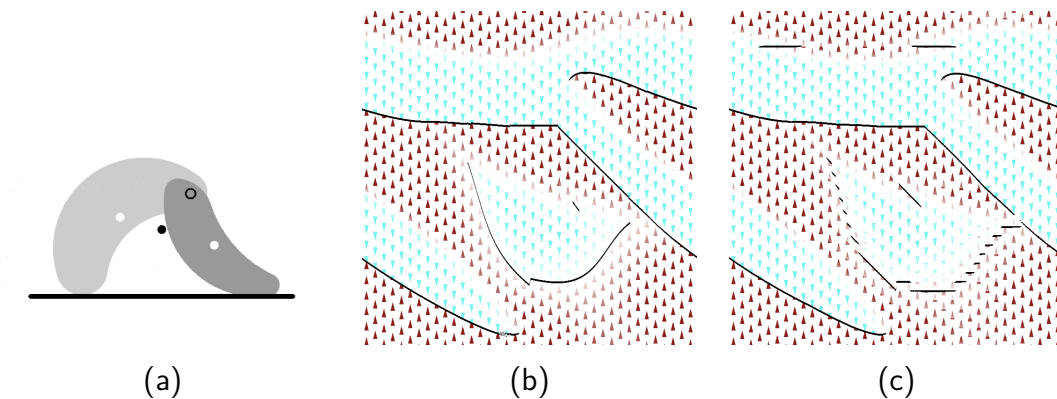


Abbildung 11: Die Mannigfaltigkeit einer Kreisbogendarstellung (b) im Vergleich mit einer Polygondarstellung (c). Der Beispielroboter (a) heißt Bananabot.

mit Shadertoy *Bananabot as Polygons*³).

Es fällt auf, dass die u-förmige Mannigfaltigkeit in der Mitte der Diagramme sich in (b) als kontinuierliche Kurve zeigt, während (c) dort unstetige Treppentufen aufweist. Hier liegt der Roboter auf der langen konvexen Kante einer seiner zwei Körperteile auf. Die Stufen entstehen durch ein diskretes Kippen von einer Ecke des Polygons zur nächsten. Das Erhöhen der Auflösung würde also die Unstetigkeit der Linie nur weiter erhöhen, anstatt den Effekt zu verringern. Des Weiteren beweisen die waagerechten Strecken oben in (c), dass die diskrete Darstellung anfälliger für „falsche“ stabile Posen ist, bei denen der Roboter angeblich auf einer kurzen Kante balanciert.

4.2 Generierung der Mannigfaltigkeit

Wie lässt sich aus der Roboterstruktur $\mathcal{R}$ also die Mannigfaltigkeit berechnen und diese als ein gerichteter Graph abspeichern? Dies erfolgt über vier Teilschritte: (1) Um eine Pose des Roboters auf Stabilität zu prüfen, wird ein Algorithmus namens POSTURIZE gebraucht, der für eine Winkelkonfiguration die Auflagepunkte und den Massenschwerpunkt der dazugehörigen Pose findet. (2) Mithilfe dieses Algorithmus kann der gesamte Posenraum nach stabilen Posen durchsucht werden, die, wenn gefunden, als Knoten

³<https://www.shadertoy.com/view/7l2Xzw>

4 KONZEPT

$v \in V$ des Graphen abgespeichert werden. (3) Sobald alle Knoten gefunden sind, werden die Kanten erstellt, um Nachbarschaft zu erzeugen und Unstetigkeit aufzudecken. (4) Zum Schluss werden alle kontinuierlich verbundenen Knoten einer Untermannigfaltigkeit zugeordnet. Der übrig bleibende Graph $\mathcal{M}$ ist dann ein diskretes Model der morphologischen Mannigfaltigkeit:

$$\begin{aligned}\mathcal{M} &= (V, E) \\ v &= (\varphi_1, \dots, \varphi_n) \in V \\ e &= (v \rightarrow u) \in E\end{aligned}$$

Die einzelnen Schritte werden im Folgenden genauer beschrieben.

4.2.1 Massenschwer- und Auflagepunkte einer Pose

Ziel des POSTURIZE-Algorithmus ist es, die Auflagepunkte und den Massenschwerpunkt einer Pose zu bestimmen. Die Auflagepunkte sind die Stellen, mit denen der Körper des Roboters den Boden berührt. Im Englischen werden sie als *points of contact* bezeichnet, deshalb die gelegentliche Abkürzung als poc. Genauso kürzt sich der Massenschwerpunkt nach der englischen Bezeichnung *center of mass* mit com ab. Algorithmus 1 beschreibt den Vorgang als Pseudoquelltext.

Die Eingangsargumente sind zum Einen die Roboterstruktur $\mathcal{R}$ und zum Anderen eine Winkelliste $\Phi = \{\varphi_1, \varphi_2, \dots, \varphi_n\}$. Die Liste Φ muss genauso viele Winkel enthalten, wie $\mathcal{R}$ Körperteile besitzt, damit für jedes Teil eine Drehung vorgegeben ist. Der erste Winkel in Φ ist der Orientierungswinkel $\varphi_1 = \varphi_o$, welcher die Drehung des gesamten Roboters relativ zum Boden angibt. Hier ist zu beachten, dass der Index von φ_o ein kleines „o“ für Orientierung und keine Null „0“ ist. Um Verwechslungen zu vermeiden, beginnen die Indizes bei 1 anstatt bei 0. Die restlichen Winkel in Φ sind Gelenkwinkel und geben die jeweils relative Drehung eines Körperteils zu seinem Elternteil an.

Für jeden Teil τ_i muss seine geometrische Form $\mathcal{F}_i$ an die richtige Stelle transformiert werden. Die Transformation hängt von seiner eigenen Drehung φ_i sowie von der Drehung aller Elternteile zwischen ihm und dem Wurzelteil $\mathcal{W}$ ab. Auch $\mathcal{W}$ ist durch ein symbolisches Gelenk mit der Welt verbunden. Dieser Gelenkwinkel entspricht der

Algorithmus 1 POSTURIZE Roboterstruktur $\mathcal{R}$ mit Winkelliste Φ

```

1: poc1 ← poc2 ← (0, ∞)
2: com ← (0, 0)
3: for each  $i \in [1, n]$  do
4:    $\mathcal{F} \leftarrow$  Kreisbogenform von  $\tau_i$ 
5:   TRANSFORMIERE  $\mathcal{F}$  um  $\varphi_i$  und Gelenkposition von  $\tau_i$ 
6:   while  $\tau_i \neq \mathcal{W}$  do
7:      $i \leftarrow$  Index des Elternteils von  $\tau_i$ 
8:     TRANSFORMIERE  $\mathcal{F}$  um  $\varphi_i$  und Gelenkposition von  $\tau_i$ 
9:   end while
10:  UPDATECOM durch  $\mathcal{F}$ 
11:  UPDATEPOCs durch  $\mathcal{F}$ 
12: end for

```

Orientierung φ_o und beeinflusst jeden Körperteil. So wird ab Zeile 6 der Stammbaum rekursiv entlang geklettert und $\mathcal{F}$ transformiert, bis der Algorithmus bei $\mathcal{W}$ angelangt ist. Eine Transformation verfrachtet $\mathcal{F}$ vom aktuellen Koordinatensystem in das System des Elternteils. Nach Zeile 9 befindet sich die Form im globalen Koordinatensystem. Die Transformation einer Kreisbogenform $\mathcal{F}$ wird in Algorithmus 6 im Anhang genauer beschrieben.

Mit UPDATECOM wird anschließend über die transformierten Massenpunkten c_i der einzelnen Körperteile der Masseschwerpunkt des kompletten Roboters com berechnet. In den Roboterabbildungen ist com immer als schwarzer Punkt eingezeichnet ist. Er berechnet sich als der Mittelwert aus den mit der Masse m_i gewichteten c_i :

$$\text{com} = \frac{\sum_{i=1}^n m_i c_i}{\sum_{i=1}^n m_i}$$

Das Finden der pocs erfordert allerdings einen eigenen Algorithmus. Bei einer stabilen Pose sind die Auflagepunkte immer auf einer Höhe. Durch diskretes Abtasten der Winkel ist es jedoch sehr unwahrscheinlich, dass genau die richtige Pose getroffen wird. Es gibt also meistens eine Höhendifferenz zwischen poc₁ und poc₂. Der Punkt mit der kleinsten y-Koordinate, also der tiefste Punkt der Form, wird als poc₁ gespeichert.

4 KONZEPT

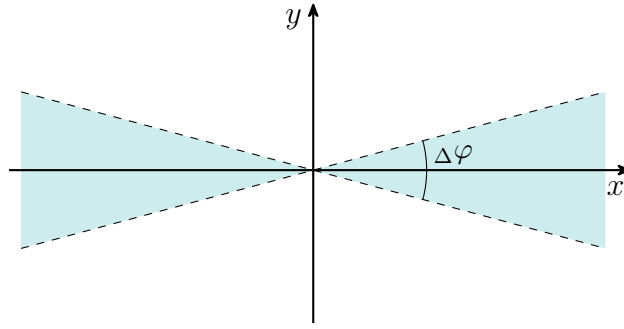


Abbildung 12: Die türkis gefärbte Fläche ist die Fehlerregion $\chi(0,0)$ des UPDATEPOCS-Algorithmus. Um als zweiter Auflagepunkt infrage zu kommen, muss sich ein Punkt p innerhalb dieses Bereiches um den ersten Auflagepunkt herum befinden: $q \in \chi(\text{poc}_1)$. Der Fehlerwinkel $\Delta\varphi$ entspricht einem diskreten Winkelschritt.

Dieser ist damit der „wirkliche“ Auflagepunkt. Mit poc_2 versucht UPDATEPOCS einen zweiten Auflagepunkt zu schätzen. Damit ein Punkt q für poc_2 infrage kommt, muss er sich in einer von der Auflösung abhängigen Fehlerregion $\chi(p)$ mit $p = \text{poc}_1$ befinden:

$$\chi(p) = \left\{ (x, y) \in \mathbb{R}^2 \mid 0 < |x - p_x| \arctan\left(\frac{\Delta\varphi}{2}\right) - |y - p_y| \right\}$$

Die Fehlerregion $\chi(p)$ ist die türkisfarbene Fläche aus Abbildung 12, insofern sich p im Ursprung befindet. $\Delta\varphi$ entspricht dabei der Schrittweite zwischen zwei diskreten Orientierungswinkeln. Je höher die Auflösung, desto kleiner ist die Fehlerregion. Ein poc_2 wird gebraucht, um den Abstand der Auflagepunkte zu berechnen. Dabei handelt es sich um eine wichtige Eigenschaft für zum Beispiel Vorwärtsbewegung, bei welcher der Abstand der Auflagepunkte periodisch vergrößert und verkleinert wird.

Zu Beginn werden poc_1 und poc_2 in Zeile 1 von POSTURIZE mit einer hohen y -Koordinate initialisiert. UPDATEPOCS, Algorithmus 2, wird dann für jedes transformierte $\mathcal{F}$ aufgerufen. Dieser prüft, ob die Form nicht vielleicht bessere pocs liefert. Das läuft in Zeile 2 durch einen Vergleich der y -Koordinaten. Wie genau die zu prüfenden Punkte einer Form bestimmt werden, führt Algorithmus 7 im Anhang aus. Es reicht jedoch an dieser Stelle zu wissen, dass eine Punktmenge von möglichen poc-Kandidaten ermittelt wird. Besitzt ein Punkt p die bisher kleinste y -Koordinate, ersetzt er in Zeile

Algorithmus 2 UPDATEPOCS durch Kreisbogenform $\mathcal{F}$

```

1: for each  $p \in \text{TIEFSTENPUNKTE von } \mathcal{F}$  do
2:   if  $p_y < \text{poc}_{1y}$  then
3:     if  $\text{poc}_1 \in \chi(p)$  then
4:        $\text{poc}_2 \leftarrow \text{poc}_1$ 
5:        $\text{poc}_1 \leftarrow p$ 
6:     else
7:        $\text{poc}_2 \leftarrow \text{poc}_1 \leftarrow p$ 
8:     end if
9:   else if  $\text{poc}_1 = \text{poc}_2$  und  $p \in \chi(\text{poc}_1)$  then
10:     $\text{poc}_2 \leftarrow p$ 
11:   else if  $\angle(\text{poc}_1, p) < \angle(\text{poc}_1, \text{poc}_2)$  then
12:     $\text{poc}_2 \leftarrow p$ 
13:   end if
14: end for

```

5 den aktuellen Wert von poc_1 . Die Funktion

$$\angle(p, q) = \arctan\left(\frac{q_y - p_y}{|q_x - p_x|}\right)$$

berechnet den Steigungswinkel von p nach q . Falls zwei Punkte für poc_2 infrage kommen, wird in Zeile 11 der Punkt mit dem kleineren Winkel zu poc_1 gewählt. Kann gar kein sinnvoller poc_2 gefunden werden, liefert der Algorithmus dank Zeile 7 $\text{poc}_1 = \text{poc}_2$. Am Ende von POSTURIZE sind also mindestens ein Auflagepunkt und der Massenschwerpunkt bekannt. Wie im dynamischen System aus Gleichung 1, kann die Fallrichtung der Pose durch das Vergleichen der x-Koordinaten von poc_1 und com bestimmt werden. Bei der Fallrichtung bzw. der Orientierungswinkelgeschwindigkeit φ'_o handelt es sich um die entscheidende Eigenschaft, zur Prüfung der Stabilität einer Pose.

4.2.2 Stabile Posen als Knoten speichern

Jeder Knoten des Mannigfaltigkeitsgraphen soll einer stabilen Pose entsprechen. Eigentlich existieren auf der kontinuierlichen Mannigfaltigkeit unendlich viele dieser Posen, da die Gelenkwinkel beliebig fein eingestellt werden können. Um sie in einem gerichteten Graphen abzuspeichern, ist man jedoch dazu gezwungen, sich auf eine diskrete Anzahl an Knoten zu beschränken. Die Winkeldimensionen φ_i der Pose werden mit einer bestimmten Auflösung n_i abgetastet. Der Abstand $\Delta\varphi_i$ zwischen zwei diskreten Winkelabtastungen auf der Dimension i beträgt dementsprechend:

$$\Delta\varphi_i = \frac{2\pi}{n_i} \quad \text{bzw.} \quad \frac{\alpha_{\max} - \alpha_{\min}}{n_i - 1} \quad (2)$$

Wenn das Gelenk nicht auf ein Intervall $[\alpha_{\min}, \alpha_{\max}]$ beschränkt ist, gilt der volle Winkelbereich von 2π . Die Anzahl aller möglichen diskreten Posen ist das Produkt aus den Auflösungen n_i jeder Winkeldimension.

Durch die Abtastung wird die genaue stabile Pose jedoch wahrscheinlich verfehlt. Stattdessen wird der Knoten, welcher sich am nächsten an der Stabilität befindet, genommen. Um die Nähe zu einer stabilen Pose zu bestimmen, wird pro Abtastung POSTURIZE wie in Abbildung 13 zweimal aufgerufen: Einmal wird in (a) der Orientierungswinkel um $\Delta\varphi_1/2$ erhöht und einmal in (b) um denselben Betrag verringert. Fallen diese beiden Posen wie in der Abbildung zueinander, existiert dazwischen eine stabile Pose, der die

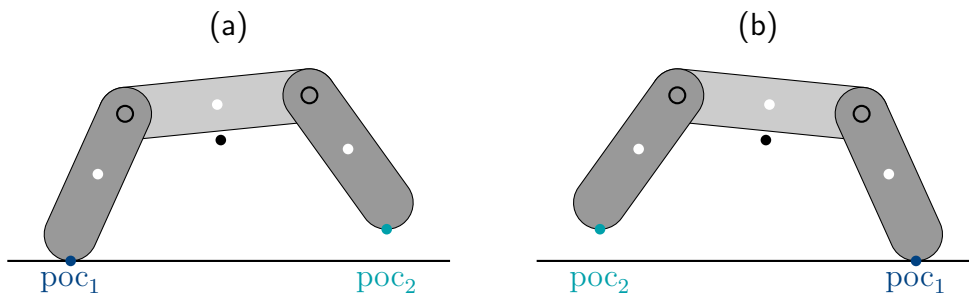


Abbildung 13: Der Orientierungswinkel wird in (a) leicht erhöht, während er in (b) leicht verringert wurde. Fallen die Posen zueinander, muss sich dazwischen eine stabile Pose befinden. Während poc_1 der wirkliche Auflagepunkt der beiden Posen ist, ist poc_2 ein geschätzter zweiter Auflagepunkt. Je höher die Auflösung von φ_0 , desto besser ist die Schätzung.

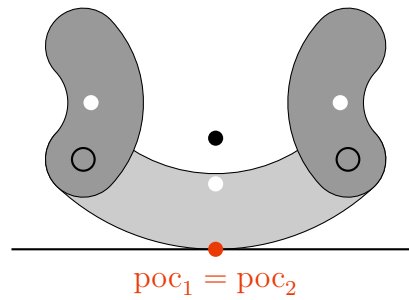


Abbildung 14: Eine gekrümmte Version von Tablebot. In dieser stabilen Pose gibt es nur einen Auflagepunkt. Der Schwerpunkt befindet sich genau senkrecht darüber.

aktuelle Abtastung am nächsten kommt. Diese kann also als Knoten gespeichert werden. Der Orientierungswinkelfehler, der durch die Abtastung entsteht, lässt sich nun auch zum Teil über den Steigungswinkel zwischen poc_1 und poc_2 korrigieren:

$$\varphi_o^* = \varphi_o - \arctan\left(\frac{poc_{2y} - poc_{1y}}{poc_{2x} - poc_{1x}}\right)$$

Es sei an dieser Stelle erwähnt, dass die Schätzung von poc_2 bei geringen Auflösungen von φ_o an Qualität verliert und deshalb den Rechenaufwand von hohen Auflösungen nicht ersetzen kann.

Eine weitere Anmerkung ist, dass vor allem bei Kreisbogenformen eine stabile Pose nicht immer zwei $pocs$ haben muss. Ein Roboter könnte stabil auf einer runden Kante liegen und seine Extremitäten vom Boden abheben wie in Abbildung 14. POSTURIZE würde für diesen Fall $poc_1 = poc_2$ produzieren. Diese Posen sind besonders interessant für dynamische Bewegungen, da hier durch eine geschickte Verlagerung des Massenschwerpunktes eine Schaukelbewegung angeregt werden kann.

4.2.3 Nachbarn mit Kanten verbinden

Nachdem der komplette Posenraum abgetastet wurde, hat man die Knotenmenge V vollständig definiert. Nun ist es an der Zeit, die Knoten miteinander zu verbinden. Die Posen, die durch eine Motoransteuerung untereinander erreichbar sind, sollten durch eine Kante zu Nachbarn werden. Abbildung 15 verdeutlicht, wie die Verbindungen zwischen den Knoten aussehen sollen. Der Roboter hat pro Freiheitsgrad zwei mögliche

4 KONZEPT

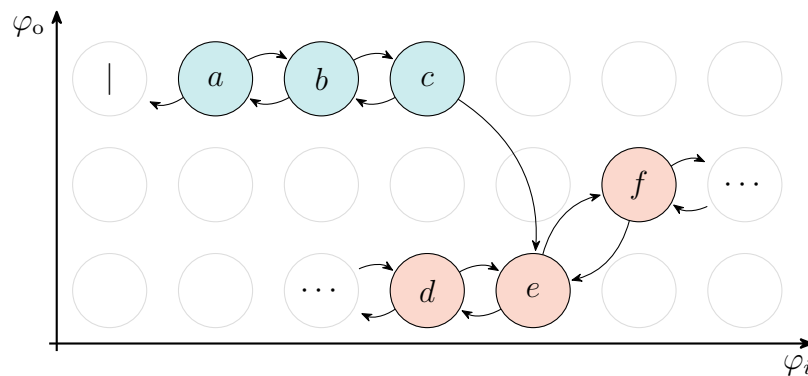


Abbildung 15: Eine schematische Darstellung, wie die Knoten verknüpft werden sollen. Die Kreise stellen mögliche diskrete Winkelkombinationen dar. Jedoch sind nur die farbigen als Knoten abgespeichert. Der Orientierungswinkel ist entlang der y-Achse aufgetragen, während die x-Achse einen beliebigen Gelenkwinkel verkörpert. Der Knoten mit der „|“-Markierung stellt eine physikalische Einschränkung dar und „ $\dots$ “-Knoten deuten an, dass hier die Verbindungen beliebig weiter gehen.

Richtungen, in die er sich bewegen kann: eine Links- oder eine Rechtsdrehung des Gelenks. Also hat jeder Knoten pro Gelenkachse φ_i zwei Nachbarknoten, die es zu finden gilt. Dabei gibt es drei verschiedene Kantentypen:

1. *Bewegend.* Der Normalfall. Hierbei handelt es sich um eine harmlose Bewegung, die der Roboter auch wieder rückgängig machen kann. Die Bewegung gilt also in beide Richtungen. Dazu gehören die Kanten $a \rightarrow b$, $b \rightarrow a$ und $d \rightarrow e$, aber auch $e \rightarrow f$ und so weiter.
2. *Fallend.* Anders als *Bewegend*-Kanten sind diese Bewegungen nicht mehr rückgängig zu machen. Es handelt sich also um einseitige Verbindungen wie $c \rightarrow e$, die kein inverses Äquivalent $e \rightarrow c$ besitzen. Durch diese Bewegung wird die Pose plötzlich instabil und fällt in eine andere Untermannigfaltigkeit.
3. *Eingeschränkt.* Eine Bewegung, die physikalisch nicht möglich ist. Gründe dafür wären Gelenkanstöße oder eine Selbstkollision. Es handelt sich also gar nicht um eine Verbindung zu einem anderen Knoten, sondern um einen Platzhalter. Diese „losen“ Kanten wie $a \rightarrow |$ teilen dem Robot mit, dass er hier nicht weiterkommt.

Die Kantentypen *Fallend* und *Eingeschränkt* stellen die Ränder von Untermannigfaltigkeiten $\mathcal{U}$ dar. *Bewegend*-Kanten hingegen befinden sich immer innerhalb von $\mathcal{U}$.

Es müssen also alle Knoten $v \in V$ in einen Algorithmus gefüttert werden, welcher v 's Nachbar $u_{i,d}$ in Richtung der Gelenkdimension i und der Bewegungsrichtung d findet. Dabei indiziert i , welcher Winkel der Pose angepasst werden soll und $d \in \{1, -1\}$ gibt die Richtung an. $d = 1$ bedeutet eine Links- und $d = -1$ eine Rechtsdrehung. Die Anpassung des ersten Winkels φ_1 entspricht keiner Gelenkbewegung, sondern einer Drehung des gesamten Roboters, da es sich dabei um den Orientierungswinkel handelt. Algorithmus 3 beschreibt den Ablauf dieses FINDENACHBAR-Prozesses.

Ausgehend von Knoten v wird ein Schritt in die gegebene Richtung gewagt:

$$\text{STEP}_{i,d} : v = (\varphi_1, \dots, \varphi_n) \mapsto u = (\psi_1, \dots, \psi_n)$$

$$\psi_j = \begin{cases} (\varphi_j + d \cdot \Delta\varphi_j) \bmod 2\pi, & j = i \\ \varphi_j, & j \neq i \end{cases}$$

Je nach dem Wert von d addiert oder subtrahiert die STEP-Funktion eine Schrittgröße vom Winkel φ_i . Die anderen Winkel bleiben unverändert. Die Schrittgröße $\Delta\varphi_i$ wird laut

Algorithmus 3 FINDENACHBAR von Knoten v in Dimension i und Richtung d

```

1:  $u \leftarrow \text{STEP}_{i,d}(v)$ 
2: if  $u$  ist unmöglich then
3:   return Kein Nachbar, Kantentyp: Eingeschränkt
4: else if  $u \in V$  then
5:   return  $u$ , Kantentyp: Bewegend
6: else ▷  $u$  ist zwar möglich, aber instabil
7:    $\hat{d} \leftarrow$  Vorzeichen der Fallrichtung nach POSTURIZE( $u$ )
8:   while  $u \notin V$  do
9:      $u \leftarrow \text{STEP}_{1,\hat{d}}(u)$ 
10:  end while
11: end if
12: return  $u$ , Kantentyp: Bewegend

```

4 KONZEPT

Gleichung 2 durch die Auflösung bestimmt. Der Modulo sorgt dafür, dass Schritte, die über den Bereich $[0, 2\pi)$ hinauswandern, auf der anderen Seite wieder hereinkommen und verkörpert dadurch die Periodizität der Winkel. Bei Gelenkanschlüssen wird der Modulo weggelassen.

Ist die neue Pose u unmöglich, beispielsweise aufgrund von überschrittenen Winkelanschlüssen, endet der Algorithmus frühzeitig in Zeile 3 und notiert eine *Eingeschränkt*-Kante. Diese Bewegung hat keinen Nachbarn. Die Kante $a \rightarrow |$ aus Abbildung 15 würde diesen Fall bedienen.

Ist die Bewegung aber möglich und es lässt sich ein passender Knoten $u \in V$ finden, endet der Algorithmus stattdessen in Zeile 5. u wird über eine *Bewegend*-Kante der Nachbar von v . Gilt jedoch $u \notin V$, handelt es sich bei u um eine instabile Pose, deren Orientierung angepasst werden muss. Dieser Fall tritt ein, wenn beispielsweise in Abbildung 15 der rechte Nachbar von c gesucht wird: Ein Schritt entlang φ_i tritt ins Leere. Um den richtigen Nachbarn e zu finden, muss φ_o angepasst werden. Die Anpassrichtung $\hat{d}$ finden wir durch das Aufrufen von `POSTURIZE` und den anschließenden Vergleich von `com` und `poc` nach dem Vorbild von Gleichung 1. Nun wird der Orientierungswinkel von u in Zeile 9 solange angepasst, bis u der Pose eines Knotens in V entspricht. Dieser ist der gesuchte Nachbar. Auch hier wird der Kantentyp *Bewegend* vorgemerkt, obwohl einige dieser Bewegungen vom Typ *Fallend* sind. Zu diesem Zeitpunkt würden die Kanten $c \rightarrow e$ und $e \rightarrow f$ vom Typ her noch nicht zu unterscheiden sein. Diese Differenzierung wird getroffen, sobald alle Kanten gefunden sind. Anders als *Fallend*-Kanten, müssen *Bewegend*-Kanten immer zweimal vorkommen: als $v \rightarrow u$ und nochmal als $u \rightarrow v$. Gibt es für eine Kante kein inverses Äquivalent, wird sie zum Typ *Fallend* geändert.

4.2.4 Zuordnung einer Untermannigfaltigkeit

Die Knoten und Kanten wurden alle gefunden. Der nächste Schritt unterteilt die Knoten von $\mathcal{M}$ in Untermannigfaltigkeiten $\mathcal{U} \subset V$. Während die Kanten nur eine Aussage über lokale Nachbarschaften treffen, entscheidet die Zugehörigkeit zu einer Untermannigfaltigkeit, ob zwei Posen ohne umzufallen, untereinander erreichbar sind. Es existieren

zumeist mehrere Untermannigfaltigkeiten. Damit jeder Knoten einer $\mathcal{U}$ angehört, muss die vereinigte Menge aller Untermannigfaltigkeiten $\hat{\mathcal{U}}$ am Ende alle Knoten beinhalten:

$$\hat{\mathcal{U}} = \mathcal{U}_1 \cup \mathcal{U}_2 \cup \dots \cup \mathcal{U}_m \stackrel{!}{=} V$$

Gilt für einen Knoten $v \notin \hat{\mathcal{U}}$, wurde dieser noch keiner Untermannigfaltigkeit zugeordnet, und er gilt als *einsam*. Zu Beginn trifft das auf alle Knoten zu.

Gegen die Einsamkeit der Knoten wird Algorithmus 4 eingesetzt. Er verwendet eine Datenstruktur namens *Warteschlange* oder *queue* im Englischen. Es lassen sich durch PUSH Elemente hinten an eine Schlange anreihen und durch POP vorne wieder entfernen. Die Warteschlange Q beachtet dabei immer die Reihenfolge *First In, First Out*, also werden die zuerst angereihten Elemente auch zuerst bedient.

Beim ersten Durchlauf ist $Q = \emptyset$ leer und es wird in Zeile 4 ein einsamer Knoten v in der Knotenliste gesucht. Die Variable i verfolgt den aktuellen Untermannigfaltigkeit-

Algorithmus 4 FINDEZUGEHÖRIGKEITEN in Knotenliste V

```

1:  $i \leftarrow 0$ 
2: while  $\hat{\mathcal{U}} \neq V$  do                                ▷ Sind noch einsame Knoten vorhanden?
3:   if  $Q = \emptyset$  then
4:      $v \in V \setminus \hat{\mathcal{U}}$                                 ▷ Suche einsamen Knoten in  $V$ .
5:      $i \leftarrow i + 1$ 
6:      $\mathcal{U}_i \leftarrow \{v\}$                                 ▷  $v$  ist der erste Knoten in  $\mathcal{U}_i$ .
7:   else
8:      $v \leftarrow Q.\text{POP}$ 
9:   end if
10:  for each  $e = (v \rightarrow u) \in \text{Nachbarkanten von } v$  do
11:    if  $e = \text{Bewegend}$  und  $u \notin \hat{\mathcal{U}}$  then
12:       $\mathcal{U}_i \leftarrow \mathcal{U}_i \cup \{u\}$                         ▷ Füge  $u$  der Menge  $\mathcal{U}_i$  bei.
13:       $Q.\text{PUSH}(u)$ 
14:    end if
15:  end for
16: end while

```

4 KONZEPT

index. Sie wird auf eins gesetzt und v wird in Zeile 6 als erster Knoten einer neuen Untermannigfaltigkeit vermerkt. Alle Nachbarknoten u von v , die über eine *Bewegend*-Kante verbunden sind, und die selbst noch einsam sind, werden in Zeile 12 jetzt derselben Untermannigfaltigkeit wie v zugeordnet und an Q angehängt. Verbindungen über andere Kantentypen liefert keine neuen Knoten für $\mathcal{U}_i$ und werden ignoriert.

Derselbe Vorgang wird nun der Reihe nach mit den Einträgen in Q wiederholt, bis die Warteschlange wieder leer ist. Dann sind alle zugehörigen Knoten dieser einen Untermannigfaltigkeit gefunden. Falls V jetzt noch einsame Knoten enthält, gehören sie zu einer neuen Untermannigfaltigkeit. i wird ein weiteres Mal um eins erhöht und ein neues $\mathcal{U}_i$ erstellt. Der Prozess wiederholt sich so lange, bis keine einsamen Knoten mehr übrig sind und $\hat{\mathcal{U}} = V$ gilt.

Dieser Algorithmus implementiert die *Breitensuche*, oder auf Englisch als *breadth first search* (BFS) bezeichnet, welche ein Verfahren zum Durchlaufen von Graphen ist. Am Ende haben alle Knoten eine Zugehörigkeit erlangt, und die Anzahl und Größe der Untermannigfaltigkeiten sind bekannt.

4.3 Suche nach Bewegungsabläufen

Nach den vier Schritten Pose finden, Knoten finden, Kanten finden und Zugehörigkeit finden, ist nun ein vollständiger, gerichteter Graph entstanden, der die morphologische Mannigfaltigkeit des Roboters repräsentiert. Dieser Graph kann als Grundlage für Algorithmen verwendet werden, die auf der Mannigfaltigkeit Bewegungsabläufe ausfindig machen.

Algorithmus 5 artikuliert einen iterativen Optimierungsprozess, der einen Mannigfaltigkeitsgraphen nach bestimmten Posen durchsucht. Die Idee ist, dass der Algorithmus an einem zufälligen Startknoten v_0 beginnt und diesen mit seinen unmittelbaren Nachbarn u vergleicht. Der Vergleich wird über eine *Fitnessfunktion* f geregelt, die die Eigenschaften einer Pose nach bestimmten Kriterien bewertet und dem Knoten eine Fitness $f(v)$ zuteilt. Für den nächsten Schritt ist dann der beste Nachbar der Ausgangsknoten. Der Algorithmus endet, wenn v der beste Knoten unter seinen Nachbarn ist. Es handelt sich bei v dann um ein *lokales Fitnessmaximum*.

Algorithmus 5 BESTEPOSE ab Startknoten v_0 mit Fitnessfunktion f

```

1:  $v \leftarrow v_0$ 
2: while  $v$  nicht Bester unter seinen Nachbarn do
3:   for each  $u \in \text{Nachbarn von } v$  do
4:     if  $f(u) > f(v)$  then
5:        $v \leftarrow u$ 
6:     end if
7:   end for
8: end while
9: return  $v$ 

```

Für eine geschmeidige Fortbewegung ist es von Vorteil, dass die aktuelle Untermannigfaltigkeit nicht verlassen wird, denn das Verlassen hängt immer mit einem Fallen und dadurch mit einem potenziellen Schaden zusammen. Damit der Algorithmus auf seiner Untermannigfaltigkeit bleibt, können Randknoten direkt mit einer schlechten Fitness bewertet werden.

Die Eigenschaften der Pose, die mit in die Fitnessfunktion einfließen, können dabei beliebig gewählt und kombiniert werden. Die zwei hier verwendeten Eigenschaften sind der Abstand der Auflagepunkte d und die Verlagerung des Schwerpunktes w . Der Abstand der Auflagepunkte wird über

$$d = \frac{\|\text{poc}_1 - \text{poc}_2\|}{d_{\max}}$$

definiert. Er wird mit der maximalen Länge des Roboters $d_{\max}$ normiert, damit sich der Wert grob in dem Intervall $[0, 1]$ befindet. Bei $d \approx 1$ wären beide Beine maximal vom Körpermittelpunkt weggestreckt und vermutlich liegt der Roboter dann flach auf seinem Bauch. Zieht er seine Beine so weit, wie es geht, zusammen erhält man $d \approx 0$. Die Gewichtsverlagerung kann mit

$$w = \frac{\text{com}_x - x_1}{|\text{poc}_{1x} - \text{poc}_{2x}|}$$

beschrieben werden. x_1 ist dabei die x-Koordinate des linken Auflagepunktes. Befindet sich der Schwerpunkt com senkrecht über dem linken Auflagepunkt, ist $w = 0$. Umge-

4 KONZEPT

kehrt gilt $w = 1$, wenn com senkrecht über dem rechten Auflagepunkt ist. Bei $w = 1/2$ befindet sich der Schwerpunkt genau oberhalb der Mitte und das Gewicht verteilt sich gleichmäßig auf beide Beine.

Um eine simple Vorwärtsbewegung zu erzeugen, reicht es, periodisch drei Schlüsselposen zu durchlaufen. Ein zum Illustrationszweck manuell erstellter Bewegungsablauf für Tablebot wird in Abbildung 16 vorgestellt. Von (a) nach (b) öffnet Tablebot seine Beine, behält aber einen Großteil seines Gewichts auf dem linken Bein. Dadurch rutscht das leichtere rechte Bein vorwärts über den Boden. Von (b) nach (c) verlagert er sein Gewicht nach rechts, behält dabei aber den Beinabstand weitgehend gleich. In der letzten Bewegung von (c) nach (a) zieht er die Beine zusammen. Diesmal ist das linke Bein leichter und wird an das rechte herangezogen. Durch ein Wiederholen dieser drei Vorgänge entsteht so eine Art „Vorwärtsschlurfen“.

Definiert man die Fitnessfunktion durch

$$f(d, w) = a_0 + a_1 d + a_2 w + a_3 dw + a_4 d^2 + a_5 w^2, \quad (3)$$

können durch geschicktes Einstellen der Koeffizienten a_i die Posen belohnt werden, deren Eigenschaften denen von (a), (b) und (c) entsprechen. Tabelle 1 zeigt auf, welche Koeffizienten sich in der Evaluation bewährt haben. Die Koeffizienten sind so gewählt, dass die berechneten f nicht größer als 1 werden und dass Eingänge im Bereich $[0, 1]$ in etwa auch Ausgänge im Bereich $[0, 1]$ erzeugen.

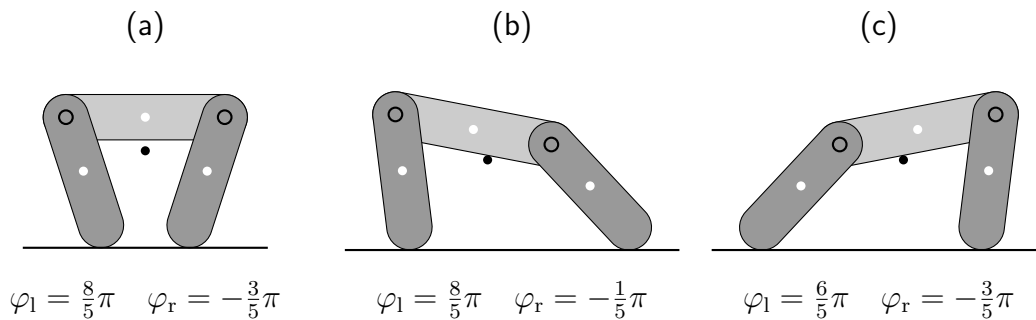


Abbildung 16: Drei Schlüsselposen von Tablebot. Ein periodisches Durchlaufen der Posen versetzt den Roboter in Vorwärtsbewegung. Die Posen ergeben sich durch ein Abwägen von Auflagepunkt Abstand d und Gewichtsverlagerung w und wurden von Hand eingestellt.

Tabelle 1: Koeffiziententabelle für die Fitnessfunktionen

	a_0	a_1	a_2	a_3	a_4	a_5
$f_{(a)}$	0,125	0,964	2,699	1,273	-2,379	-3,081
$f_{(b)}$	0,4	1,075	0,168	-0,214	-0,514	-0,614
$f_{(c)}$	-0,046	0,861	1,061	0,214	-0,514	-0,614

Losgelassen auf einen Mannigfaltigkeitsgraphen findet der Algorithmus also die Pose mit maximaler Fitness. Eine Schwäche hat dieses Vorgehen allerdings: Es findet nur lokale Maxima, also Posen, die in ihrer unmittelbaren Umgebung die besten sind. Außerhalb dieser Regionen könnten sich jedoch noch geeignetere Posen befinden. Damit der Algorithmus diese findet, darf der Startknoten also nicht ganz zufällig gewählt werden, sondern muss bereits in der Nähe von und auf derselben Untermannigfaltigkeit wie der beste Knoten starten. Ein genauerer Blick auf die Mannigfaltigkeit von Tablebot, Abbildung 6 auf Seite 16, zeigt, in welchen Regionen sich geeignete Startknoten befinden können. Damit Tablebot maximale Kontrolle über seine Poseneigenschaften d und w hat, sollten beide Gelenke φ_l und φ_r einen aktiven Einfluss auf die Orientierung φ_o haben. Dieser Einfluss zeigt sich durch eine starke Wölbung der Untermannigfaltigkeit in Richtung beider Gelenkdimensionen.

Der Ausschnitt der türkisfarbenen Untermannigfaltigkeit aus Abbildung 6, auf den diese Beschreibung zutrifft, ist in Abbildung 17 dargestellt. Es handelt sich dabei um genau diejenigen Posen, bei denen beide Beine von Tablebot auf dem Boden aufliegen und den hellgrauen Körper in der Luft halten. Die drei von Hand eingestellten Schlüsselposen aus Abbildung 16 befinden sich allesamt innerhalb des gezeigten Bereichs.

Senkrecht von oben betrachtet wird dieser Ausschnitt zu einem zweidimensionalen *Konfigurationsraum*. Dort werden die Knoten nur noch in Abhängigkeit von den Gelenkwinkeln betrachtet. Abbildung 18 auf Seite 42 verdeutlicht den Konfigurationsraum mit fünf verschiedenen Einfärbungen. Die Fitnessfunktion, welche sich oberhalb des jeweiligen Diagramms befindet, bestimmt die Farbe eines Knotens. Eine Fitness von

4 KONZEPT

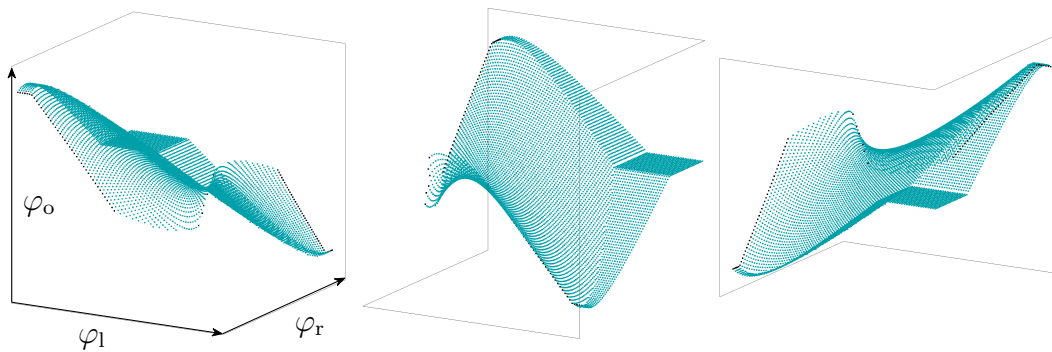


Abbildung 17: Ein Teilbereich der türkisfarbenen Untermannigfaltigkeit aus Abbildung 6 mit $-1 \leq \varphi_o \leq 1$ und $2,135 \leq \varphi_l \leq 5,760$ und $-2,618 \leq \varphi_r \leq 1,001$. Die Auflösung beträgt $n = 90$ pro Achse. Es handelt sich dabei um die Posen, bei denen Tablebot seinen hellgrauen Körper mit den Beinen in der Luft hält. Beide Gelenkbewegungen haben starken Einfluss auf die Körperorientierung φ_o .

$f = 0$ wird türkis eingefärbt und rot bedeutet eine Fitness von $f = 1$. Für Werte dazwischen ist die Farbe linear interpoliert, weshalb die gräulich, braunen Regionen in etwa bei $f = 1/2$ liegen. Die rechte und untere Außenkante des Konfigurationsraumes wird durch Winkelanschlüsse beschränkt. Oben und links geht die Mannigfaltigkeit weiter, liegt aber außerhalb des interessanten Bereichs mit starker Wölbung. Die Einkerbung in der unteren rechten Ecke entsteht teilweise durch Instabilität, also Umfallen in eine andere Untermannigfaltigkeit, und teilweise durch Selbstkollision, wenn die Beine des Roboters aneinanderstoßen. Unstetigkeiten im Farbverlauf spiegeln hier die Falten und Knicke der Mannigfaltigkeit wider. Die drei Schlüsselposen aus Abbildung 16 sind als weiße Punkte in den Diagrammen eingetragen.

Die zwei oberen Konfigurationsräume zeigen die zwei Eigenschaften d und w ohne besondere Verrechnung. Für $f(x) = d$ erhält man einen fast linearen Gradienten von rechts unten nach links oben. Schlüsselpose (a) hält sich im türkisfarbenen Bereich dieses Diagramms auf, da dort d gering gehalten wird. Das $f(x) = w$ -Diagramm zeigt, dass in der Mitte des Ausschnitts die Masse auf beide Beine gleichmäßig verteilt ist, aber an den Rändern der Mannigfaltigkeit auf eine der Seiten verlagert ist. Diese

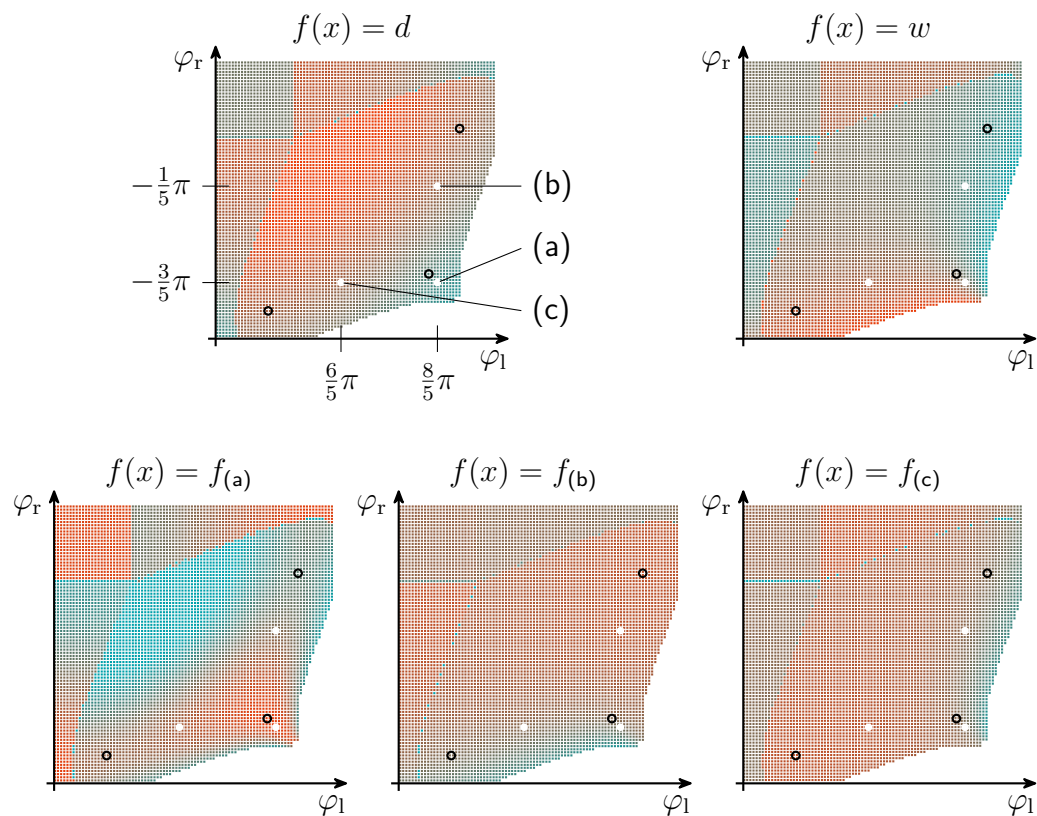


Abbildung 18: Fünfmal derselbe Konfigurationsraum, jeweils eingefärbt durch eine andere Fitnessfunktion. Dabei entspricht türkis einer Fitness von 0 und rot einer Fitness von 1. Die weißen Punkte markieren die von Hand eingestellten Schlüsselposen aus Abbildung 16 und die schwarzen Kreise zeigen die algorithmisch gefundenen Schlüsselposen aus Abbildung 19.

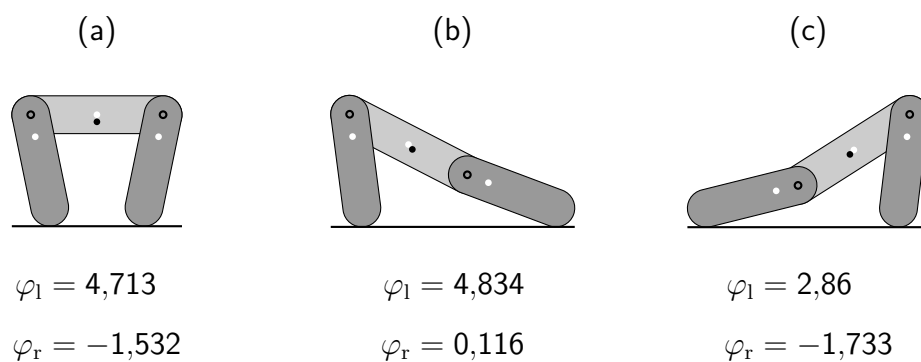


Abbildung 19: Algorithmisch ermittelte Schlüsselposen von Tablebot

4 KONZEPT

Bereiche sind für (b) und (c) geeignet.

Die unteren drei Konfigurationsräume in Abbildung 18 verwenden nun die Fitnessfunktionen mit den Koeffizienten aus Tabelle 1. Die als schwarze Kreise eingezeichneten Posen hat Algorithmus 5 jeweils als Maximum einer der drei Fitnessfunktionen identifiziert. Somit sind sie anders als die Posen aus Abbildung 16 algorithmisch und nicht von Menschenhand bestimmt. Besagte Posen sind in Abbildung 19 illustriert. Es ist also möglich, Bewegungsabläufe durch Schlüsselposen zu definieren, die dank der Fitnessfunktionen algorithmisch und für beliebige Morphologien bestimmt werden können.

4.4 Robotersteckkit

Es wird nun ein flexibles Hardwaresystem gebraucht, mit welchem sich die entworfenen Morphologien und gefundenen Bewegungsabläufe schnell in Betrieb nehmen und evaluieren lassen. Diesen Job übernimmt das modulare Steckkit LARA. Der Name stammt von „moduLARA Roboter“. Die Kreisbogenformen können als steckbare Module 3D-gedruckt und zu einem Roboter zusammengesteckt werden. Die aus der Mannigfaltigkeit gewonnenen Bewegungsabläufe können dadurch schnell und unkompliziert an einem realen Model getestet und ausgewertet werden.

LARA formuliert folgende Designprinzipien:

1. *Druckbarkeit*: Module und Verbindungsteile werden 3D-gedruckt und sind für diesen optimiert. Das heißt, es werden keine Stützstrukturen gebraucht, Sollbruchstellen vermieden und Druckzeiten und Filamentbedarf reduziert.
2. *Steckbarkeit*: Der Aufwand bei Montage und Demontage bleibt gering. Wo es geht, verzichtet LARA auf Schrauben, Werkzeug und Kugellager oder verwendet stattdessen ein Steck- und Klemmsystem.
3. *Modularität*: Alle Module sind mit allen anderen Modulen kombinierbar. Der Morphologie des Roboters wird dabei weiterhin möglichst viel Freiraum gegeben.
4. *Simplizität*: Die Anzahl der verschiedenen Einzelteile, die eine Inbetriebnahme eines Roboters ermöglichen, sind auf ein Minimum reduziert. Standardteile wie

die Steckverbinder lassen sich für andere Morphologien wiederverwenden und erfordern keinen Neudruck.

5. *Eckenlos*: Die Module sollen dem Anwender gefallen. In Harmonie mit den Kreisbögen verzichtet LARA auf spitze Ecken und gerade Kanten und arbeitet stattdessen mit vorwiegend runden Konturen.

In Abbildung 20 sind einige der Bauteile exemplarisch abgebildet. Zwei ausgedruckte Kreisbogenformen wie in (a) werden zu einem Körperteil beziehungsweise einem Modul zusammengesteckt. Mit den eckigen Steckverbindern aus (b) werden beide Hälften zusammengehalten. Die Verbinder können mit der Hand eingesetzt und wieder entfernt werden und erfordern dadurch kein Werkzeug für die Montage. Sie lassen sich innerhalb von fünf Minuten drucken und können bei einer neuen Morphologie wiederverwendet werden.

Zwei Robotermodule hängen durch ein motorisiertes Gelenk zusammen. Als Motoren dienen die *Dynamixel XL330* von *Robotis*. Um weiterhin den Montageaufwand gering zu halten, klemmt LARA auch die Motoren ein, anstatt sie festzuschrauben. Die Motoren passen spielfrei in optionale Klemmhalterungen wie in (c) hinein. Das einzige Teil, welches geschraubt wird, ist die Steckverbindung, die am Rotor befestigt ist. (d) zeigt die beiden verbundenen Motormodulhälften. Auf der anderen Seite der Drehachse befindet sich ein passives Gelenk. Spezielle Verbinder – halb eckig, halb rund – gestatten den rotatorischen Freiheitsgrad und vermeiden den Bedarf für ein Kugellager. In (e) ist ein aus zwei Modulen bestehender Roboter zusammengebaut worden. LARA implementiert ein Schichtsystem. Module der inneren Schicht beinhalten die Motoren, an denen Module der äußeren Schicht befestigt werden. An einer anderen Stelle des äußeren Moduls kann dann ein weiteres motorisiertes Innenmodul angebracht werden und so weiter. Jedes Modul verfügt über freie Steckverbindungen, wodurch die Anwender:in flexibel Klemmhalterungen für Batterien, Akkus, Sensoren oder Prozessoren anbringen kann. In (f) wurde dadurch ein Steckverbinder zur Kabelführung umfunktioniert.

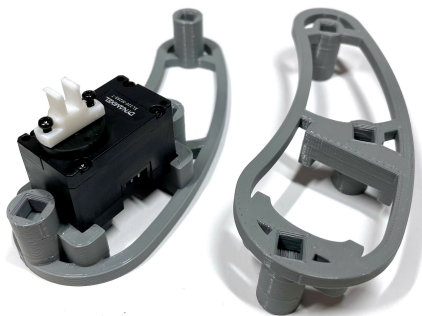
4 KONZEPT



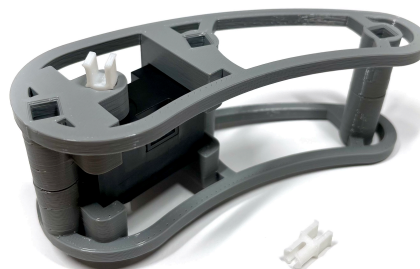
(a)



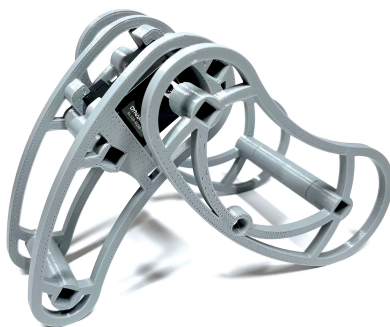
(b)



(c)



(d)



(e)



(f)

Abbildung 20: Darstellung essentieller Einzelteile des LARA-Steckkits

5 Evaluation

Dieses Kapitel untersucht die Tauglichkeit des gesamten Gestaltungsprozesses von Entwurf über Analyse bis hin zum Einsatz mit dem LARA-Steckkit. Dazu werden zwei Versuchsreihen an der parametrisierbaren Morphologie von Tablebot durchgeführt.

Die Performanz einer Vorwärtsbewegung lässt sich am verlässlichsten über die zurückgelegte Strecke S und die dafür benötigte Energie E messen. Je mehr S und je weniger E ein Verhalten leistet, desto besser ist seine Energieeffizienz S/E . Beides wird pro Versuch nach einer festen Anzahl von Bewegungszyklen geprüft. Diese Größen sind abhängig von einer Reihe an Variablen. Die zwei Versuchsreihen messen die Abhängigkeit der Größen S und E zum einen von bestimmten Eigenschaften der Schlüsselposen und zum anderen von verschiedenen morphologischen Parametern. Andere Variablen wie Materialeigenschaften von Boden beziehungsweise Roboterkörper und Motorantriebsgeschwindigkeit bleiben dabei konstant.

Welche drei Schlüsselposen in einer Bewegung durchlaufen werden, bestimmt der Algorithmus 5, welcher durch die Fitnessfunktion aus Gleichung 3 beeinflusst wird. Die Koeffizienten dieser Funktion können so eingestellt werden, dass bestimmte Eigenschaften einer Pose bevorzugt werden, nämlich der Auflagepunktabstand d und die Gewichtsverlagerung w . Die erste Versuchsreihe testet die Energieeffizienzen von vier Bewegungsabläufen auf einer zunächst konstant bleibenden Robotermorphologie. Die Abläufe resultieren aus der Kombination verschiedener Fitnessfunktionen.

Anschließend gilt es in Versuchsreihe 2, die beste Funktionskombination auf verschiedenen Morphologien zu testen. Die Morphologien sind vier Versionen von Tablebot mit unterschiedlichen Beinmodulen. Die Formparameter dieser Module sind dabei möglichst divers im Parameterraum verstreut. Für jede der Morphologien wird eine Mannigfaltigkeit erstellt und ein Bewegungsablauf generiert. Jeder gefundene Bewegungsablauf wird dann auf jeder Morphologie evaluiert. Das Ziel dieser Reihe ist weniger die Abhängigkeit der Laufbewegung von der Morphologie zu untersuchen, als zu zeigen, wie gut die algorithmischen Bewegungsabläufe auf ihre Morphologie zugeschnitten sind, oder ob sich die Abläufe auf beliebigen Morphologien als ähnlich effizient erweisen.

5.1 Versuchsaufbau

Die Roboterplattform ist eine ausgedruckte Version von Tablebot, wie in Abbildung 21 dargestellt. Als Mikroprozessor findet der *OpenCM9.04* von *Robotis* seinen Einsatz. Über eine Micro-USB-Schnittstelle lässt sich der Prozessor mit der *Arduino IDE* programmieren. An den Gelenken befindet sich jeweils ein *Dynamixel XL330-M288-T*-Motor ebenfalls von *Robotis*. Über vier 1,5 V AA-Batterien, die in Serie geschaltet sind, wird eine Versorgungsspannung von $U_B \approx 6\text{ V}$ geliefert. Die Körperteile bestehen aus 3D-gedrucktem, grauem PLA, welches nach eigener Messung eine Dichte von etwa 9 g/cm^3 besitzt. Die Geometrie der Körperteile aus der Abbildung ist mit *CASHEW*-Formen modelliert, deren Parameter in Tabelle 2 eingetragen sind. Dieser Roboter wird für Versuchsreihe 1 eingesetzt. In Versuchsreihe 2 werden die Parameter der Beinmodule verändert. Im ausgestreckten Zustand hat der Roboter eine Länge von etwa 39 cm. Sein Gesamtgewicht beträgt 279 g.

Ein Bewegungszyklus besteht immer aus drei Schlüsselposen, die periodisch durchlaufen werden. Pro Versuch werden immer drei solcher Zyklen durchgeführt. Die zurückgelegte Strecke S ist dabei der Abstand zwischen dem Start- und dem Endpunkt der Bewegung.



Abbildung 21: Die Tablebot-Morphologie, realisiert durch das Steckkit LARA. Die zwei Beinmodule enthalten jeweils einen Motor. Die Motoren sind durch einen rotatorischen Freiheitsgrad mit dem Torsomodul verbunden, an welchem der Prozessor und die vier Batterien befestigt sind.

Tabelle 2: Eigenschaften der einzelnen Körperteile von Tablebot

	d	$r_1 = r_2$	γ	Gewicht
Bein (je)	102,0 mm	20,4 mm	0	45 g
Torso	142,8 mm	20,4 mm	0	189 g

Die Roboter starten und beenden ihre Bewegung immer in der neutralen Pose $P = (3\pi/2, -\pi/2)$. In dieser Pose hat der Roboter beide Beine senkrecht in Richtung Boden angewinkelt.

Die Energie berechnet sich durch zeitliches Integrieren der Leistung $p(t)$. Die Leistung ist das Produkt von Motorstrom und Motorspannung: $p(t) = u(t) i(t)$. Strom und Spannung können direkt am Motor gemessen und über die Micro-USB-Schnittstelle ausgegeben werden. Die Spannung wird in Form von Pulsdauermodulation – oder auf English *puls width modulation* – geregelt, welche sich im Intervall $\text{PWM}(t) \in [0, 1]$ befindet. Die Motorspannung berechnet sich dann aus $u(t) = \text{PWM}(t) \cdot U_B(t)$. Da die Versorgungsspannung beim Antreiben der Motoren schwankt, wird auch diese in Abhängigkeit der Zeit betrachtet. Fällt die Batteriespannung vor einem Versuch unter 5,4 V, also unter 90 % der Nennspannung, werden die Batterien ersetzt. E berechnet sich schlussendlich aus der zeitlich integrierten Leistung beider Motoren:

$$E = \int p_l(t) + p_r(t) dt$$

Die Abtastfrequenz der gemessenen Größen beträgt etwa 19 Hz.

Da für die Messung der Energie der Roboter mit einem Kabel verbunden werden muss, welches die Laufbewegung beeinflusst, werden E und S in separaten Versuchen aufgenommen. Unbehandelte mitteldichte Holzfaserverplatten (MDF) dienen den Versuchen als Boden, um gleichmäßige Reibungseigenschaften zu garantieren. Die maximale Motordrehzahl ist auf 6,87 /min eingestellt.

5.2 Versuchsreihe 1

In dieser Versuchsreihe werden vier verschiedene Bewegungsabläufe an der klassischen Tablebot-Morphologie getestet. Die Schlüsselposen der Bewegungsabläufe sind in Abbildung 22 zu sehen. Ein Zyklus besteht aus drei Posen der Sorte (a), (b) und (c), von denen es jeweils zwei verschiedene Versionen gibt. Der Beinabstand d der Pose (a1) ist kleiner als bei Pose (a2). Für (b) und (c) muss der Beinabstand geöffnet und dabei das Gewicht einmal nach links und einmal nach rechts verlagert werden. Die Verlagerung w wird aber bei großen Beinabständen schwieriger. Dadurch wird bestimmt, ob Tablebot mehr Wert auf einen großen Beinabstand in (b1) und (c1) legen sollte oder ob eine maximale Verlagerung wie in (b2) und (c2) die Strategie der Wahl ist. Die Koeffizienten für die Fitnessfunktionen der sechs Schlüsselposen sind in Tabelle 3 aufgeführt. Tabelle 4 kombiniert diese Posen zu vier verschiedenen Bewegungsabläufen B1 bis B4. Jeder Ablauf wurde vierzehnmal an Tablebot getestet. Die ersten sieben Versuche geschahen kabellos, um ein durchschnittliches S zu messen. Weitere sieben Versuche, diesmal mit einem Kabel, bestimmten ein gemitteltes E der Bewegung. Die Ergebnisse sind in Abbildung 23 zu sehen. Die Beinbreite von Pose (a) zu verringern sowie diese in (b) und (c) zu vergrößern, verbesserte das Ergebnis von S um ~ 50 mm. Hinsichtlich des

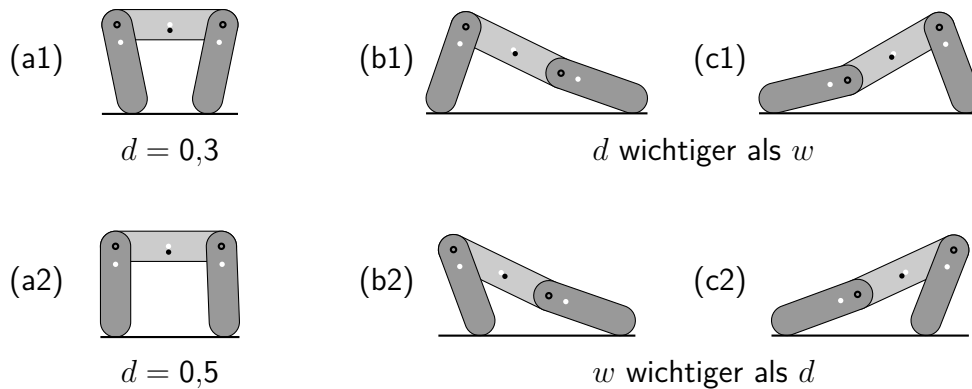


Abbildung 22: Sechs Schlüsselposen der Fortbewegung. (a1) hat einen kleineren Beinabstand als (a2). (b1) und (c1) legen mehr Wert auf eine große Schrittweite, während (b2) und (c2) mehr Gewicht verlagern.

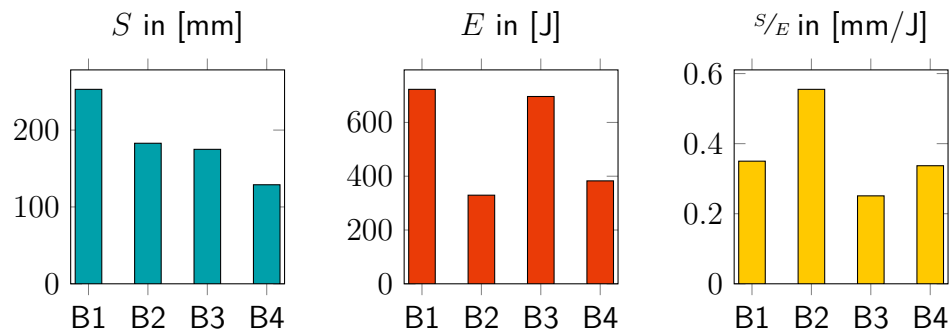


Abbildung 23: Balkendiagramme der ersten Versuchsreihe. Das linke Diagramm zeigt die durchschnittlich zurückgelegte Strecke pro Zyklus des Bewegungsablaufs. Daneben in Rot ist der gemittelte Energieverbrauch eines Zyklus dargestellt. Das dritte Diagramm gibt die Energieeffizienz s/E eines Verhaltens an. B2 liegt klar vorn.

Tabelle 3: Fitnesskoeffiziententabelle für Versuchsreihe 1

	a_0	a_1	a_2	a_3	a_4	a_5
$f_{(a1)}$	0,125	0,964	2,699	1,273	-2,379	-3,081
$f_{(a2)}$	-0,056	2,212	2,112	2,176	-3,4	-3,2
$f_{(b1)}$	0,2	1,475	0,068	-0,114	-0,714	-0,314
$f_{(c1)}$	-0,046	1,361	0,561	0,114	-0,714	-0,314
$f_{(b2)}$	0,4	1,075	0,168	-0,214	-0,514	-0,614
$f_{(c2)}$	-0,046	0,861	1,061	0,214	-0,514	-0,614

Tabelle 4: Posenkombinationen für die Bewegungsabläufe

B1	(a1)	(b1)	(c1)
B2	(a1)	(b2)	(c2)
B3	(a2)	(b1)	(c1)
B4	(a2)	(b2)	(c2)

5 EVALUATION

E -Diagramm bewiesen sich Posen (b2) und (c2) mit fast halb so viel Energieverbrauch wie (b1) und (c1). Verhalten B2 kombiniert die Vorteile beider Messwerte und erlangte damit ein mindestens 159 % besseres S/E als die anderen Verhaltensweisen.

Als Ergebnis der Reihe kann Folgendes festgehalten werden. Durch ein Variieren der d und w -Parameter können unterschiedliche Bewegungsstrategien entworfen werden. Wenn man nur an Strecke interessiert ist, bietet sich ein Sprint-artiges Verhalten wie B1 an. Allerdings zeigten die Messungen von S/E , dass kleinere, gut verlagerte Schritte effizienter sind als größere, energieaufwendige Schritte.

Zwischen den beiden Strategien kann der Roboter situationsbedingt wechseln. Zum Beispiel würde er standardmäßig eine energieeffiziente Fortbewegung wie B2 bevorzugen, um möglichst weite Strecken zurücklegen zu können und seinen Batteriestand zu schonen. In einer Gefahrensituation, die ein Fluchtverhalten erfordert, kann er aber mit Verhalten B1 in einen Sprint wechseln, um kurzfristig schneller voranzukommen.

5.3 Versuchsreihe 2

Diese Versuchsreihe testet das Verhalten B2 aus Versuchsreihe 1 mit den vier unterschiedlichen Robotermorphologien aus Abbildung 24. Sie unterscheiden sich nur anhand der CASHEW-Formparameter ihrer Beinmodule. Das hellgraue Torsomodul der

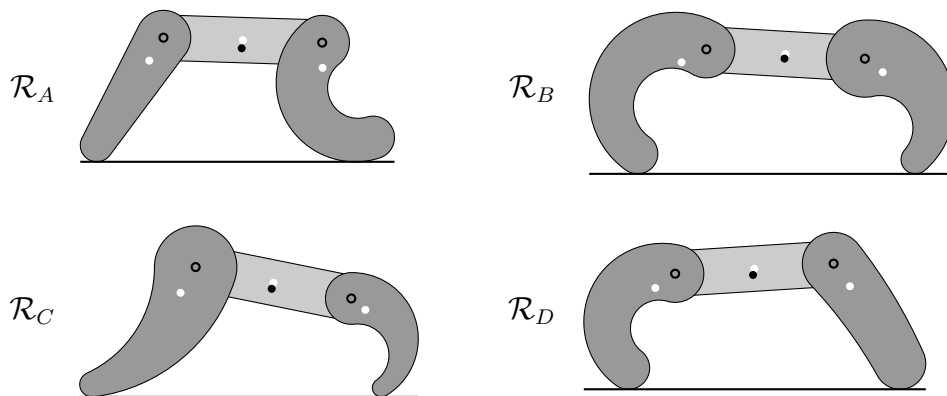


Abbildung 24: Vier neue Morphologien für Versuchsreihe zwei. Es handelt sich um Versionen von Tablebot, jedoch haben die Beine möglichst diverse morphologische Eigenschaften.

vier neuen Morphologien gleicht dem von Tablebot. Da dieses Modul nur wenig Einfluss auf das Laufverhalten ausübt, wird es für alle Versuche wiederverwendet. Druck- und Montageaufwand von neuen Torsomodulen wird dadurch gespart.

Bei dem abgebildeten Parametersatz handelt es sich um den *diversesten* nach tausend zufällig erstellten Sätzen. Die Diversität Δ berechnet sich wie folgt: Angenommen der Vektor $X = (x_1, x_2, \dots, x_8)$ repräsentiert die acht Parameter der zwei Beinformen eines Roboters. Dann gilt für den Abstand zwischen zwei Parametervektoren:

$$\|X - Y\| = \sqrt{\sum_{i=1}^8 (x_i - y_i)^2}$$

Die Längen d und Radien r sind immer so skaliert, dass die maximale Länge des Roboters gleich bleibt. Das garantiert die Vergleichbarkeit der Morphologien. Vor der Berechnung müssen die Parameter mit $x_i = (x_i^* - \mu_i)/\sigma_i$ normiert werden, damit jeder Parameter gleich stark in Δ einfließt. Dabei ist μ_i der Mittelwert und σ_i die Standardabweichung der Normalverteilungen, die die Parameter x_i^* zufällig generieren. Tabelle 5 listet die verwendeten μ und σ auf. Fällt ein Parameter außerhalb des physikalisch Umsetzbaren wie zum Beispiel negative Werte, wird neu gewürfelt. Die Versuchsreihe testet vier Roboter gegeneinander, weshalb ein Parametersatz aus vier solcher normierten Parametervektoren besteht $\{X_1, \dots, X_4\}$. Die Diversität Δ eines Satzes berechnet sich dann aus:

$$\Delta = \sum_{j=1}^4 \sum_{i=1}^4 \|X_i - X_j\|$$

Für jede Morphologie $\mathcal{R}_i$ wird eine Mannigfaltigkeit erstellt und dann mithilfe der Fitnessfunktionen von B2 ein passender Bewegungsablauf B_i ermittelt. In Bezug auf

Tabelle 5: Mittelwert μ und Standardabweichung σ der verwendeten Normalverteilungen

	d in [mm]	$r_{1,2}$ in [mm]	γ in [rad]
μ	102,0	20,4	0
σ	40,8	20,4	$\pi/2$

5 EVALUATION

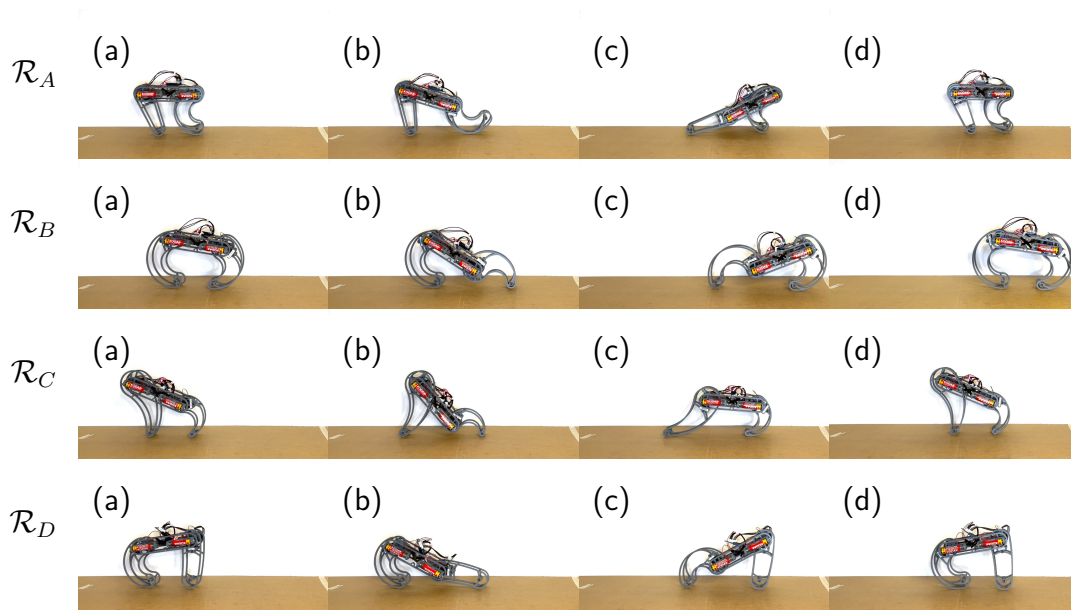


Abbildung 25: Ausgewählte Standbilder aus Videoaufnahmen von Versuchsreihe 2. Jedes Standbild zeigt eine der Schlüsselposen, wobei Pose (d) dieselbe Pose wie (a) ist, nur einen Zyklus später.

Abbildung 24 ist die Laufbewegung immer von links nach rechts ausgerichtet, weshalb das linke Bein auch als Hinter- und das rechte als Vorderbein bezeichnet wird. Abbildung 25 zeigt Standbilder von Videoaufnahmen der Morphologien jeweils mit ihrem dazugehörigen Bewegungsablauf. Jeder Ablauf wird nun mit jeder Morphologie fünfmal nach S und weitere fünfmal nach E getestet.

Die durchschnittlichen Ergebnisse jedes Versuchs sind in Tabelle 6 bis 8 abgebildet. Je stärker die Färbung einer Tabellenzelle, desto besser schnitt der Versuch ab. Da Verhalten mit wenig E gesucht sind, erhalten in Tabelle 7 kleinere Werte ein tieferes Rot. Die Zeilen der Tabellen repräsentieren die Morphologien und die Spalten die verwendeten Laufverhalten. Dadurch treffen die passenden Paare auf der Hauptdiagonalen aufeinander. Zellen mit einem „x“ bedeuten, dass dieses Verhalten auf der Morphologie instabil ist und von sinnvollen Messungen abhält. Auf $\mathcal{R}_A$ hat kein anderer Bewegungsablauf funktioniert. Das lag daran, dass das flache Hinterbein zu weit eingeklappt wird, sodass der Schwerpunkt hinter den Auflagepunkt verschoben wird und der Roboter auf

sein „Gesäß“ fällt. Für $\mathcal{R}_B$ wurde der energieeffizienteste Bewegungsablauf gefunden. Hingegen erwiesen sich die für $\mathcal{R}_C$ und $\mathcal{R}_D$ gefunden Bewegungen nur am zweitbesten. Als Ergebnis von Versuchsreihe 2 kann Folgendes festgehalten werden. Der Vorgang garantiert für morphologieunabhängig ein stabiles Laufverhalten zu bestimmen. Hingegen kann das Verhalten mit einem Effizienzoptimum noch nicht verlässlich ermittelt werden.

Tabelle 6: Strecke S in [mm]

	B_A	B_B	B_C	B_D
$\mathcal{R}_A$	180,00	x	x	x
$\mathcal{R}_B$	177,00	215,93	107,53	161,27
$\mathcal{R}_C$	119,17	178,93	137,40	128,00
$\mathcal{R}_D$	159,60	170,13	75,93	171,40

Tabelle 7: Energie E in [J]

	B_A	B_B	B_C	B_D
$\mathcal{R}_A$	281,87	x	x	x
$\mathcal{R}_B$	322,03	380,15	256,16	415,31
$\mathcal{R}_C$	387,05	402,98	338,21	390,65
$\mathcal{R}_D$	369,08	443,47	319,12	429,66

Tabelle 8: Energieeffizienz s/E in [mm/J]

	B_A	B_B	B_C	B_D
$\mathcal{R}_A$	0,639	x	x	x
$\mathcal{R}_B$	0,550	0,568	0,420	0,388
$\mathcal{R}_C$	0,308	0,444	0,406	0,328
$\mathcal{R}_D$	0,432	0,384	0,238	0,399

6 Fazit

Als in den 1960er Jahren der Begriff *künstliche Intelligenz* entstanden ist, ging man davon aus, dass Intelligenz auf reiner Denkkraft beziehungsweise Rechenleistung beruhe und irgendwo tief im Gehirn verankert sei. Alle Aspekte von intelligentem Verhalten seien algorithmisch zu beschreiben und könnten dadurch von einem Computer simuliert werden. Die Befürchtung, dass Maschinen eine dem Menschen überlegene künstliche Intelligenz entwickeln, schien hinsichtlich des exponentiellen technischen Wachstums immer wahrscheinlicher zu werden. 60 Jahre später sind die Ausmaße dieser Entwicklungen bereits zu spüren: 1964 entwickelte J. Weizenbaum den ersten Chatbot namens *ELIZA*, 1997 besiegte *Deep Blue* den amtierenden Schachweltmeister Garri Kasparov, und 2020 wurde *GPT3* veröffentlicht, ein Sprachverarbeitungsmodell mit 175 Milliarden Trainingsparametern.

Allerdings scheinen alle aufgezählten Systeme trotz beeindruckender Leistungen eine gemeinsame Schwäche zu haben: Sie glänzen nur bei einer bestimmten Aufgabe. Die Ergebnisse sind schwierig allgemein anzuwenden und können nicht adäquat auf Störungen in ihrer Aufgabenumgebung reagieren. Ist das wirklich Intelligenz? Wie ist es mit Konzepten wie Seele, Bewusstsein und sozialen Strukturen? Jeder Mensch kann sich unter diesen Begriffen etwas vorstellen, dennoch sind sie künstlich noch nicht nachmodelliert worden. Der klassische Ansatz scheint sich hier in einer Sackgasse zu befinden.

Klassische neuronale Netze gelten als die Methode der Wahl, um komplexe Probleme zu lösen. Wochenlang werden sie mit zurechtgeschnittenen Trainingsdaten gefüttert und ihre Parameter angepasst und optimiert. Doch auch diese Netze versagen bei Eingangssignalen, die vom vorliegenden Datensatz abweichen, und erfordern neues Training. Auch in der modernen Robotik ist dieses Phänomen zu beobachten. Oftmals wird ein Roboter genau für eine bestimmte Aufgabe konstruiert und ihm die gewünschten Bewegungen oder Verhaltensweisen aufwendig und mit viel Ingenieurskunst einprogrammiert. Doch ein Schritt außerhalb der Testumgebung, ein unerwarteter Reiz und der Roboter kann sich nicht helfen. Die Hoffnung ist, dass Roboter für die Altenpflege,

die Raumfahrt oder die Bergung von Verletzten in Katastrophengebieten eingesetzt werden. Es handelt sich dabei um komplexe Aufgaben in unbekannten, unstrukturierten Umgebungen, die man nur schwierig oder sogar gar nicht im Voraus modellieren kann. Dadurch ist es umso schwieriger eine Maschine genau dafür zu entwickeln.

Ein Blick auf ein evolutionär entstandenes Lebewesen zeigt, dass Intelligenz anders funktionieren kann. Ein Neugeborenes folgt weder einem vorgegebenen Ablaufplan, noch ist sein Körper vollständig entwickelt. Am Anfang würde man Babys nicht als intelligent bezeichnen. Aber trotzdem entfalten sie schon früh komplexe Verhaltensweisen wie Greifen, Krabbeln und Schreien und entwickeln sich im Laufe des Lebens durch ein aktives Interagieren mit der Umwelt immer weiter[16]. Menschen wie Blinde oder Taubstumme finden sich trotz ihrer sensorischen Beeinträchtigungen in der Welt zurecht, besitzen aber grundverschiedene Weltbilder und Verhaltensweisen. Bereits 1915 zeigten Marinas Experimente mit Affen[17], dass das Gehirn nicht nur unter anderen Umständen zurechtkommt, sondern sich sogar umprogrammieren kann, wenn bestimmte sensorische oder motorische Gegebenheiten umgepolt werden. Ganz im Gegensatz zum klassischen Verständnis, erweckt es den Eindruck, als resultiere intelligentes Verhalten kontinuierlich aus der Interaktion mit der Umwelt und ist ein nie abgeschlossener Lernprozess zu betrachten.

Während der klassische Ansatz versucht, die intelligenten Verhaltensweisen von der physikalischen Welt zu abstrahieren, betont *Embodiment* die zentrale Rolle der Verkörperung des Gehirns. Wenn Menschen Intelligenz künstlich erzeugen wollen, müssten sie ihre Systeme mit einem Körper, also einer Morphologie, ausstatten und autonome Roboter bauen.

6.1 Zusammenfassung

Im Sinne des Embodiment war das Ziel der Arbeit einen Gestaltungsprozess für ebensolche Roboter zu entwickeln. Die Eigenschaften der Morphologie, konkreter morphologischer Mannigfaltigkeiten, werden als Grundlage für verhaltenssteuernde Algorithmen genutzt. Die Arbeit bewältigte folgende Ziele:

6 FAZIT

1. Die Kreisbogendarstellung ist ein nützliches Werkzeug, um ästhetische, geschmeidige Roboterkörper zu modellieren. Sie hat keine Einschränkungen gegenüber der Polygondarstellung und ermöglicht das Definieren von intuitiven parametrisierten Formen wie der CASHEW-Form.
2. Durch wenige, überschaubare Algorithmen kann einer Robotermorphologie eine Mannigfaltigkeit in Form eines gerichteten Graphen entnommen werden. Über einen simplen Vergleich von der x-Koordinaten vom Massenschwerpunkt und Auflagepunkt wird die Fallrichtung einer Pose ermittelt und ihre Stabilität analysiert.
3. Ein optimierender Algorithmus macht unter Verwendung von Fitnessfunktionen geeignete Schlüsselposen ausfindig, die über ein periodisches Durchlaufen Verhaltensweisen wie zum Beispiel Vorwärtsbewegung hervorrufen.
4. Diese Bewegungsabläufe und Morphologien können anschließend mit dem Steckkit LARA in Betrieb genommen und evaluiert werden. Das Steckkit besteht zum Großteil aus 3D-druckbaren Teilen und erfordert nur wenig Montageaufwand. Dadurch wird das Durchführen von Testreihen erheblich erleichtert.

Verschiedene Laufbewegungen, getestet an Tablebot, zeigten, dass eine maximale Gewichtsverlagerung beim Laufen Energie spart. Dadurch lohnt es sich, kleinere, sparsame anstatt größere, schnellere Schritte zu machen. Die CASHEW-Form und das LARA-Steckkit ermöglichten das Modellieren und Realisieren von vier zusätzlichen Robotern mit diversen morphologischen Eigenschaften. Der Prozess erstellte für jeden Roboter eine eigene Mannigfaltigkeit und ermittelte eine individuelle, stabile Vorwärtsbewegung. Die Energieeffizienz des Laufverhaltens war allerdings noch nicht optimiert.

Die Arbeit hat einige Einschränkungen. Die erforschten Morphologien befinden sich alle in einer zweidimensionalen Welt. Zwar existiert die Hardware in der dreidimensionalen realen Welt, doch auch sie verkörpert zweidimensionale Roboter. Grund für das Weglassen der dritten Dimension ist, dass die Zusammenhänge leichter zu verstehen und zu visualisieren sind. Dabei ist die Hoffnung, dass interessante Schlüsse in der vereinfachten zweidimensionalen Welt gezogen und in die dritte Dimension übertragen werden können.

Außerdem ist *Selbstkollision* für die Kreisbogendarstellung noch nicht vollständig integriert worden. Dadurch findet die Software gelegentlich auch Posen, die real gar nicht möglich sind.

6.2 Weiterführende Arbeit

Die Software besteht zurzeit aus mehreren kleinen Programmen, die jeweils einen Teil des Prozesses übernehmen. Diese in eine benutzerfreundliche Applikation zusammenzuführen, könnte ein nächster Schritt in der Software-Entwicklung sein. Dazu gehört ein geeigneter Simulator von Kreisbogenformen, welcher auch Selbstkollisionen registrieren kann. Eine passende Schnittstelle zum LARA-System könnte auch implementiert werden, die direkt die zu druckenden Module generiert und auf die Platzierung von nicht druckbaren Teilen, wie Prozessoren, Batterien und Motoren achtet.

Die Generierung der Mannigfaltigkeit passiert zurzeit nur auf der CPU. Durch parallele Berechnungen auf einer Grafikkarte könnte der Prozess beschleunigt werden. Eventuell wäre es möglich, die parametrische Veränderung in der Morphologie in Echtzeit auf einer gerenderten Mannigfaltigkeit zu visualisieren. Es ist dazu auch vorstellbar, Algorithmen zu entwickeln, die durch die Mannigfaltigkeiten direkt Zusammenhänge zwischen Morphologie und Verhaltensmöglichkeiten erkennen können und aufgrund dessen, Gestalter:innen Ratschläge zum Entwurf geben.

In dieser Arbeit befanden sich die Roboter auf einem einsamen eindimensionalen Strich, welcher sich „Boden“ nannte. Es wäre interessant zu erforschen, in welcher Weise sich Hindernisse und Unebenheiten in der morphologischen Mannigfaltigkeit kenntlich machen. Der Roboter kann eventuell diese Veränderungen nutzen, um Hindernisse zu klassifizieren und den Umgang mit ihnen zu lernen. Irgendwann muss auch der Schritt in die dritte Dimension gewagt werden.

Die Ergebnisse der Evaluation zeigten, dass die energieeffizienteste Bewegung noch nicht verlässlich gefunden wird. Zusätzliche Tests mit unterschiedlichen Fitnessfunktionen und Morphologien könnten weiterhin Abhängigkeiten zum Energieverbrauch und zur Laufstrecke offenbaren und somit die Ergebnisse des Prozesses verbessern.

Dynamische und materielle Eigenschaften blieben dabei außen vor. Das Material der

6 FAZIT

Körperteile sowie die Reibungseigenschaften des Bodens beeinflussen stark, wie viel Gewicht auf einen Fuß zu verlagern ist, damit dieser nicht wegrutscht. Ein Erhöhen der Motorantriebsgeschwindigkeit hat zur Folge, dass die Massenträgheit mit beachtet werden muss. Dadurch kann der Roboter sich temporär außerhalb seiner Mannigfaltigkeit befinden oder durch schnelle Bewegungen stabile Posen zum Umkippen bringen. Ein autonomer Roboter kann diese materiellen und dynamischen Zusammenhänge aber auch ausnutzen.

Vor allem soll die Entwicklung in Richtung Autonomie gehen. Für diese Arbeit wurde noch viel von Hand eingestellt, wie zum Beispiel die Fitnessfunktionen zum Finden von geeigneten Schlüsselposen oder die Formparameter der Morphologie. Ein wirklich autonomer Roboter sollte eigenständig lernen, nach welchen Kriterien er seine Posen bewertet und er dadurch sein Verhalten selbständig erschafft.

A Anhang: Modellieren durch Kreisbögen

Ein Kreisbogen ist ein kontinuierlicher Abschnitt eines Kreises. Kreisbögen eignen sich, um Formen mit runden Konturen und nur wenigen Parametern zu definieren. Das Kapitel führt in die Mathematik ein, die hier zum Entwerfen von Robotermorphologien verwendet wird. Gleichzeitig stellt das Kapitel die parametrisierte Kreisbogenform namens *CASHEW* vor, die ihren Namen der Ähnlichkeit zu Cashewkernen verdankt. Die abschließende Abbildung des Kapitels auf Seite 70 zeigt verschiedene Versionen dieser Form und verdeutlicht am besten die Möglichkeiten der Kreisbogendarstellung.

A.1 Mathematische Definition

Eine Kreisfläche ist eine zweidimensionale Punktmenge, die sich dadurch auszeichnet, dass alle Punkte einen Abstand zum Mittelpunkt des Kreises besitzen, der kleiner ist als der Radius. Alle Punkte, die genau den Radius r als Abstand zum Kreismittelpunkt m haben, bilden die eindimensionale Kreislinie K :

$$K = \{x, y \in \mathbb{R} \mid r^2 = (x - m_x)^2 + (y - m_y)^2\}$$

Ein Kreisbogen B ist eine kontinuierliche Teilmenge von K . Eine Möglichkeit, einen Kreisbogen B vollständig zu beschreiben, ist über folgende fünf Parameter: Startposition $p = (p_x, p_y)$, Startrichtung φ , Krümmung κ und Bogenlänge l . Die Tupel-Schreibweise wäre also:

$$B = (p_x, p_y, \varphi, \kappa, l) \subset K$$

Abbildung 26 zeigt die Bedeutungen der fünf Parameter anhand von drei Beispielbögen. Die Krümmung ist gleich dem Kehrwert des Radius: $|\kappa| = 1/r$. Anders als der Radius kann κ vorzeichenbehaftet sein. Das Vorzeichen

$$\text{sign}(\kappa) = \begin{cases} +1, & \kappa > 0 \\ 0, & \kappa = 0 \\ -1, & \kappa < 0 \end{cases}$$

A ANHANG: MODELLIEREN DURCH KREISBÖGEN

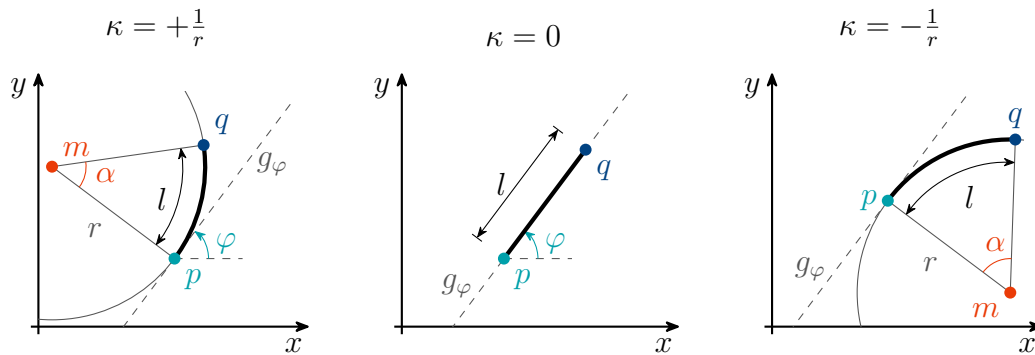


Abbildung 26: Drei verschiedene Kreisbögen und ihre Eigenschaften. Das Vorzeichen von κ gibt dabei die Krümmungsrichtung an. Der Bogen mit $\kappa = 0$ ist eigentlich keiner, da er weder einen Radius noch einen korrespondierenden Kreis vorweist. Es handelt sich dabei um eine Strecke. Startwinkel φ ist für alle drei Bögen gleich.

gibt dabei die Krümmungsrichtung an: Positiv krümmt nach links und negativ nach rechts. Wenn der Radius unendlich groß wird, wird aus dem gekrümmten Kreisbogen die geschlossene *Strecke* $[p \ q]$ mit einer Krümmung von $\kappa = 0$. Für solche „geraden Bögen“ $B_{\kappa=0}$ existiert weder ein korrespondierender Kreis noch ein Radius, weshalb sie eigentlich nicht mehr als Bögen bezeichnet werden können. Durch κ ist der Übergang von positiv gekrümmt, über ungekrümmt, bis zu negativ gekrümmt (vgl. Abbildung 26 von links nach rechts) fließend, weshalb hier Strecken zur Kreisbogendarstellung dazuzählen. Einige der folgenden Eigenschaften gelten allerdings nur für gekrümmte Bögen mit $\kappa \neq 0$ und sind mit dem Index $\kappa \neq 0$ gekennzeichnet.

Beispielsweise haben gekrümmte Bögen einen Mittelpunktswinkel

$$\alpha = l\kappa,$$

welcher für Strecken $\alpha = 0$ beträgt. Es sei mit

$$e_{\angle}(\varphi) = \begin{pmatrix} \cos(\varphi) \\ \sin(\varphi) \end{pmatrix}$$

von nun an ein Einheitsvektor gemeint, der in Richtung von φ zeigt. Dann ist die Richtungsgerade g_{φ} durch

$$g_{\varphi}(t) = p + t \ e_{\angle}(\varphi)$$

definiert. Sie entspricht der Tangente des Kreisbogens an der Startposition p . Strecken sind einfach Abschnitte auf dieser Geraden, welche bei Punkt p anfangen.

Der Mittelpunkt m des korrespondierenden Kreises K muss sich auf einer Geraden rechtwinklig zu g_φ befinden, damit der Bogen tangential aus p herauskommt. Der Abstand zwischen p und m wird durch den Radius und damit durch κ vorgegeben:

$$m_{\kappa \neq 0} = p + \frac{1}{\kappa} e_\perp \left(\varphi + \frac{\pi}{2} \right)$$

Um einen beliebigen Punkt d auf dem Bogen zu finden, der um die Bogenlänge λ von p entfernt liegt, gilt:

$$d(\lambda) = \begin{cases} g_\varphi(\lambda), & \kappa = 0 \\ m + \frac{1}{\kappa} e_\perp \left(\varphi - \frac{\pi}{2} + \lambda \kappa \right), & \kappa \neq 0 \end{cases}$$

Durch die Fallunterscheidung gilt die Funktion für alle Kreisbögen inklusive ungekrümmter Strecken. Solange $\lambda \in [0, l]$, liefert $d(\lambda)$ Punkte auf dem Bogen. Durch das Einsetzen von diskreten λ -Werten lässt sich der Bogen mit dieser Funktion abtasten und in ein Polynom umwandeln. Der Endpunkt q des Bogens kann durch ein Einsetzen der Bogenlänge l in d gefunden werden:

$$q = d(l)$$

Die mathematische Definition von B als Punktmenge ist demnach:

$$B = \{ d(\lambda) \mid \lambda \in [0, l] \}$$

Für gekrümmte Bögen gilt $l < l_{\max}$. Die maximale Bogenlänge wird durch den Umfang des korrespondierenden Kreises $l_{\max} = 2\pi/\kappa$ vorgegeben. Bogenlängen, die darüber hinausgehen, sind nicht sinnvoll. Für Strecken gibt es diese Einschränkung nicht.

In manchen Fällen ist es nützlich, einen Punkt bei einem bestimmten absoluten Winkel auf dem korrespondierenden Kreis zu finden. Zum Beispiel für das Definieren einer Bounding Box kann dann bei 0° , 90° , 180° und -90° nachgeschaut werden. Der Punkt w , der bei Winkel ψ auf dem korrespondierenden Kreis liegt, ist:

$$w_{\kappa \neq 0}(\psi) = m + \frac{1}{|\kappa|} e_\perp(\psi) \quad (4)$$

A ANHANG: MODELLIEREN DURCH KREISBÖGEN

Allerdings erreicht der Bogen den Winkel ψ nur, wenn gilt:

$$|\alpha| \geq \left(\text{sign}(\kappa)(\psi - \varphi) + \frac{\pi}{2} \right) \bmod 2\pi \quad (5)$$

A.1.1 Kreisbogenform

Ein Aneinanderreihen von mehreren Kreisbögen ergibt eine Kreisbogenform. Eine Form $\mathcal{F}$ mit n vielen Bögen B_i könnte so definiert sein:

$$\mathcal{F} = \{B_1, B_2, \dots, B_n\}$$

Abbildung 28 auf Seite 66 zeigt ein Beispiel für eine solche Form. Damit die Bögen immer tangential ineinander übergehen und keine Ecken oder Unstetigkeiten entstehen, muss die Startposition gleich der Endposition seines Vorgängers sein und der Startwinkel um den Mittelpunktswinkel erhöht werden:

$$p_{i+1} = q_i \quad (6)$$

$$\varphi_{i+1} = \varphi_i + \alpha_i \quad (7)$$

Damit die Form am Ende bündig abschließt, muss diese Eigenschaft auch vom letzten Bogen B_n zum ersten Bogen B_1 zutreffen.

Dadurch ist das Speichern einer Startposition p_i und Startwinkel φ_i für jeden Bogen B_i eigentlich überflüssig, da sie sich aus κ und l der vorherigen Bögen erschließen lassen. Es genügt, p_1 und φ_1 des ersten Bogens zu definieren. Allerdings würde man dadurch jedes Mal die Bogenkette entlang klettern müssen, um Punkte von Bögen mit $i > 1$ zu finden. Mit der Fünf-Parameter-Darstellung fällt die Entscheidung für mehr Speicheraufwand anstatt für mehr Rechenaufwand.

Um eine Form $\mathcal{F}$ durch Verschieben und Drehen räumlich zu transformieren, genügt es, p und φ eines jeden Bogens anzupassen. Die Transformation ist in Algorithmus 6 beschrieben. Als Erstes wird in Zeile 2 die Startrichtung φ um den Drehwinkel ψ vergrößert. Anschließend muss die Startposition p mit einer Drehmatrix um ψ rotiert und dann um v versetzt werden. Dieser Algorithmus wird beispielsweise von POSTURIZE auf Seite 28 benutzt.

In einigen Fällen ist es nützlich, den Punkt eines Bogens zu finden, der sich räumlich am tiefsten befindet, also die kleinste y-Koordinate hat. Beispielsweise braucht Algorithmus 2, UPDATEPOCS, pro Form eine Menge L von solchen Punkten. Dieser Vorgang ist in Algorithmus 7 beschrieben. Erst wird geprüft, ob der Bogen den Winkel $-90^\circ = -\pi/2$, also senkrecht Richtung Boden, erreicht. Jeder Bogen der Form fügt in Zeile 5 seinen tiefsten Punkt x durch $L \cup \{x\}$ der Menge bei. Strecken werden in Zeile 2 direkt ignoriert, da sie nur den Boden berühren können, wenn sie flach aufliegen und dann eigentlich jeder Punkt der Strecke gleich tief ist. In diesem Fall liefert der Vorgänger- und Nachfolgerbogen jeweils den Anfangs- und Endpunkt der Strecke, welche als Auflagepunkte genügen.

Algorithmus 6 TRANSFORMIERE Form $\mathcal{F}$ um Winkel ψ und Versatz v

```

1: for each  $B \in \mathcal{F}$  do
2:    $\varphi \leftarrow \varphi + \psi$ 
3:    $p \leftarrow \begin{pmatrix} \cos(\psi) & -\sin(\psi) \\ \sin(\psi) & \cos(\psi) \end{pmatrix} p + \begin{pmatrix} v_x \\ v_y \end{pmatrix}$ 
4: end for
```

Algorithmus 7 TIEFSTENPUNKTE von Form $\mathcal{F}$

```

1:  $L \leftarrow \emptyset$ 
2: for each  $B_{\kappa \neq 0} \in \mathcal{F}$  do
3:   if  $B$  erreicht  $-\pi/2$  nach Gleichung 5 then
4:      $x \leftarrow w(-\pi/2)$  nach Gleichung 4
5:      $L \leftarrow L \cup \{x\}$ 
6:   end if
7: end for
8: return  $L$ 
```

A.1.2 Erweiterung für Ecken

Zwar war die Motivation der Kreisbogendarstellung das Umgehen von Ecken, jedoch kann es unter Umständen nützlich sein, Ecken in eine Morphologie einzubauen. Abbildung 27 zeigt drei Ideen, Ecken mit Kreisbögen umzusetzen. (a) ist eine abgerundete Ecke. Je mehr die Krümmung erhöht und die Länge verkürzt wird, desto stärker approximiert der Bogen damit eine wirkliche Ecke. Im Grenzwert erreicht man irgendwann (b). Dieser Fall kann mit $l = 0$ abgedeckt werden. Wenn ein Bogen keine Länge mehr hat, verliert κ seine Bedeutung, und der Bogen besteht nur aus einem einzigen Punkt, nämlich p . Stattdessen kann κ bei $l = 0$ als der Eckwinkel umfunktioniert werden, um nun auch „echte“ Ecken darzustellen. Eine dritte Möglichkeit besteht darin, negative Bogenlängen zu akzeptieren. Ein Bogen mit negativer Länge läuft, wie in (c) dargestellt, die Richtungsgerade rückwärts entlang. Es entstehen dadurch krallenartige Formen, die zum Klettern oder Greifen von Nutzen sein könnten. Die mathematischen Definitionen müssen für diesen Fall nicht angepasst werden. Allerdings sind idealisierte Ecken wie (b) und (c) nur wenig realistisch. Viele Fertigungsverfahren wie beispielsweise der 3D-Druck haben einen minimalen Eckradius. Eine Approximation durch Bögen

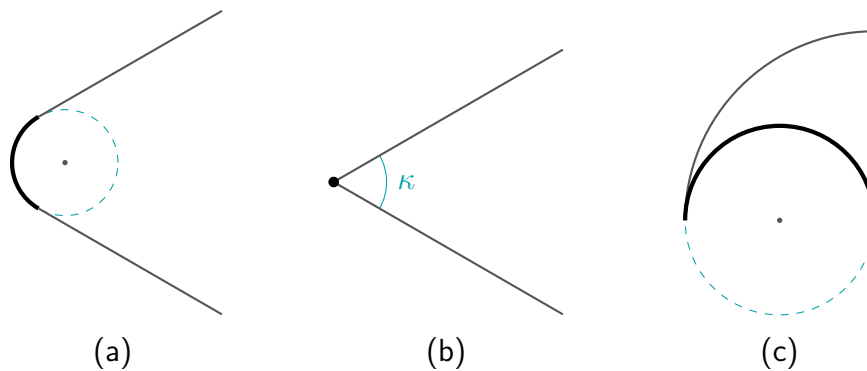


Abbildung 27: Drei Möglichkeiten, Ecken durch Kreisbögen zu erzeugen. (a) zeigt die Approximation einer Ecke durch einen kurzen Bogen mit großer Krümmung. (b) zeigt eine wirkliche Ecke. Der Eckpunkt wird ebenfalls durch einen Bogen dargestellt, für welchen $l = 0$ gilt und κ als Eckwinkel umfunktioniert wird. (c) visualisiert eine negative Bogenlänge, wodurch krallenartige Ecken entstehen.

mit positiver Bogenlänge wie in (a) ist deshalb zu empfehlen.

A.2 Cashew-Form

Die CASHEW-Form ist ein Beispiel für einen möglichen parametrisierten Körperteil. Der Name stammt von der Ähnlichkeit zum Cashewkern, siehe Abbildung 28. Die Form beginnt mit den zwei Kreisen K_1 und K_2 , deren Mittelpunkte um den Abstand d auseinander liegen. Ihre Radien sind durch r_1 und r_2 festgelegt. Diese beiden Kreise werden nun durch zwei weitere Kreise K_3 und K_4 erweitert. K_3 und K_4 liefern je einen Kreisbogen, um K_1 und K_2 einmal von oben und einmal von unten zu verbinden. Der Mittelpunktswinkel γ dieser Verbindungsbögen ist der vierte Parameter der Form. Schlussendlich lässt die korrekte Aneinanderreihung der vier Bögen eine geschlossene Kreisbogenform entstehen:

$$\text{CASHEW}(d, r_1, r_2, \gamma) = \{B_1, B_2, B_3, B_4\}$$

Zu beachten ist hier, dass die Indizes der Bögen nicht den Indizes ihrer korrespondierenden Kreise entsprechen. Die Reihenfolge der Bögen startet links bei B_1 und arbeitet

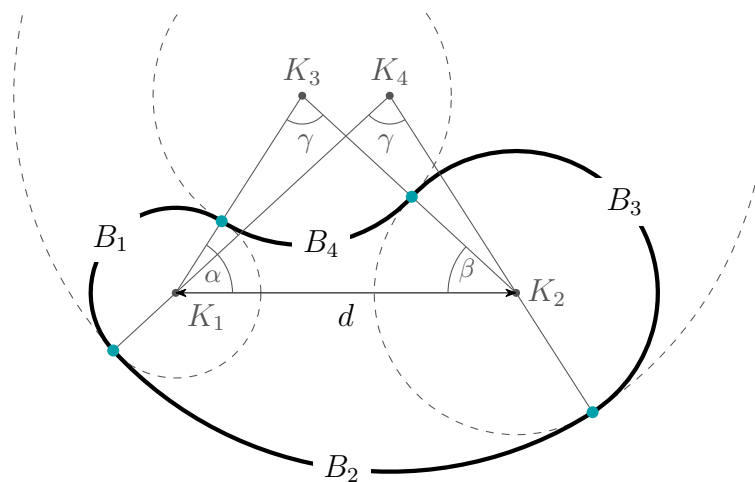


Abbildung 28: Beispiel einer CASHEW-Form. Die vier Kreise $K_{1...4}$ liefern jeweils einen Bogen $B_{1...4}$ der Form. Die türkisfarbenen Punkte verdeutlichen die Start- und Endpunkte der Bögen.

A ANHANG: MODELLIEREN DURCH KREISBÖGEN

Tabelle 9: Definitionen der Bögen bei $\gamma = 0$

B_1		$B_1 \quad B_2 \quad B_3 \quad B_4$				
φ	$\alpha + \pi/2$	κ	$1/r_1$	0	$1/r_2$	0
p	$r_1 e_{\angle}(\alpha)$	l	$2r_1\beta$	δ	$2r_2\alpha$	δ

sich dann gegen den Uhrzeigersinn um die Form. Die Übergangspunkte, welche in der Abbildung türkis markiert sind, werden durch die drei Winkel α , β und γ bestimmt. Diese Winkel entstehen, wenn aus den Kreismittelpunkten Dreiecke gebildet werden. Die zwei Dreiecke $K_1K_2K_3$ und $K_1K_2K_4$ haben dieselben Längen und Winkel, sind aber entlang der Vertikalen gespiegelt. Der lokale Koordinatenursprung befindet sich beim Mittelpunkt von K_1 , wodurch der Mittelpunkt von K_2 immer bei $(d, 0)$ liegt.

Die Aufgabe ist nun, die vier Bögen zu bestimmen, wenn nur die Parameter d , r_1 , r_2 und γ gegeben sind. Dafür wird von jedem Bogen B_i Krümmung κ_i und Bogenlänge l_i benötigt. Von den Startpositionen und -winkeln werden nur p_1 und φ_1 des ersten Bogens gebraucht. Der Rest lässt sich durch Gleichung 6 und 7 erschließen.

Zu Anfang wird eine Fallunterscheidung getroffen. Gilt $\gamma = 0$, gibt es keine verbindenden Kreise, und B_2 und B_4 sind Strecken. In diesem Fall gilt:

$$\begin{aligned}\alpha &= \arccos\left(\frac{r_1 - r_2}{d}\right) \\ \beta &= \pi - \alpha \\ \delta &= \left\| \begin{pmatrix} d \\ 0 \end{pmatrix} + (r_2 - r_1) e_{\angle}(\alpha) \right\|\end{aligned}$$

α und β bilden jetzt kein Dreieck mehr, sondern verlaufen parallel zueinander. Der Abstand zwischen dem Start- und dem Endpunkt von B_4 wird als δ bezeichnet. Tabelle 9 zeigt die Definitionen der Bögen. Links sind Startposition und -richtung des ersten Bogens angegeben und rechts die Längen und Krümmungen aller Bögen.

Gilt allerdings $\gamma \neq 0$, wird die Rechnung etwas aufwendiger. Mithilfe des Kosinussatzes ergeben sich die Radien der Verbindungskreise R_3 und R_4 als Nullstellen folgender

quadratischer Gleichung:

$$0 = R^2 + \underbrace{(r_1 + r_2)}_p R + \underbrace{\frac{r_1^2 + r_2^2 - 2r_1 r_2 \cos \gamma - d^2}{2 - 2 \cos \gamma}}_q$$

$$R_3 = -\frac{p}{2} + \text{sign}(\gamma) \sqrt{\left(\frac{p}{2}\right)^2 - q}$$

$$R_4 = +\frac{p}{2} + \text{sign}(\gamma) \sqrt{\left(\frac{p}{2}\right)^2 - q} = R_3 + p$$

Die zweite Nullstelle ergibt $-R_4$, weshalb auch $R_4 = R_3 + p$ geschrieben werden kann. Das Vorzeichen von γ entscheidet dabei, welcher der größere konvexe Bogen und welcher der kleinere konkave Bogen wird. Wenn $\text{sign}(\gamma) = +1$, dann ist B_4 wie in Abbildung 28 der konkave Bogen. In diesem Fall erzeugt die Formel negative Verbindungsradien R_3 und R_4 , mit denen ohne Vorzeichenwechsel weitergerechnet wird. Die Differenz zwischen den Verbindungsradien ist dabei immer $p = r_1 + r_2$. Nun, da die Radien der Verbindungskreise bekannt sind, kann der Mittelpunkt $m_3 = (x, y)$ des ersten Verbindungskreises K_3 bestimmt werden:

$$x = \frac{1}{2d} \left(d^2 + (R_3 + r_1)^2 - (R_3 + r_2)^2 \right)$$

$$y = \text{sign}(y) \sqrt{(R_3 + r_1)^2 - x^2}$$

Ob sich m ober- oder unterhalb der x-Achse befindet, also ob y mit einem negativen oder positiven Vorzeichen versehen wird, entscheidet $\text{sign}(y)$, nicht zu verwechseln mit $\text{sign}(\gamma)$:

$$\text{sign}(y) = -\text{sign}(\gamma \bmod 2\pi - \pi)$$

Der Mittelpunkt wird benötigt, um die Winkel α und β zu finden. m_4 wird nicht gebraucht, um fortzufahren. Aber der Vollständigkeit halber ist $x_4 = d - x_3$ und $y_4 = y_3$. Wenn die Indizes von x und y weggelassen werden, handelt es sich von nun

A ANHANG: MODELLIEREN DURCH KREISBÖGEN

an um x_3 und y_3 . Die fehlenden Winkel sind dann:

$$\alpha = \arctan(\operatorname{sign}(\gamma)y, \operatorname{sign}(\gamma)x) + \Psi$$

$$\beta = \pi - \gamma - \alpha$$

$$\Psi = \begin{cases} 2\pi, & \operatorname{sign}(\gamma) < 0 < \operatorname{sign}(y) \\ 0, & \text{sonst} \end{cases}$$

Der Korrekturwert Ψ wirkt dabei der Unstetigkeit durch die $\arctan$ -Funktion entgegen. Tabelle 10 zeigt die nötigen Parameter zur Konstruktion der Form, wenn $\gamma \neq 0$. Für Startposition und -richtung gilt dabei dieselbe Definition wie in Tabelle 9. Die Möglichkeiten dieser Form werden in der Matrix in Abbildung 29 verdeutlicht. Ein Invertieren von $\operatorname{sign}(\gamma)$ sorgt für eine Spiegelung an der x-Achse, genauso bewirkt ein Vertauschen von r_1 und r_2 eine Spiegelung entlang der Vertikalen. Schlussendlich sollte $d > (r_1 + r_2)$ gelten, also K_1 und K_2 sollten sich nicht überlappen, um sinnvolle Formen zu erzeugen.

Tabelle 10: Definitionen der Bögen bei $\gamma \neq 0$

	B_1	B_2	B_3	B_4
κ	$1/r_1$	$1/R_4$	$1/r_2$	$-1/R_3$
l	$(\gamma + 2\beta) r_1$	γR_4	$(\gamma + 2\alpha) r_2$	γR_3

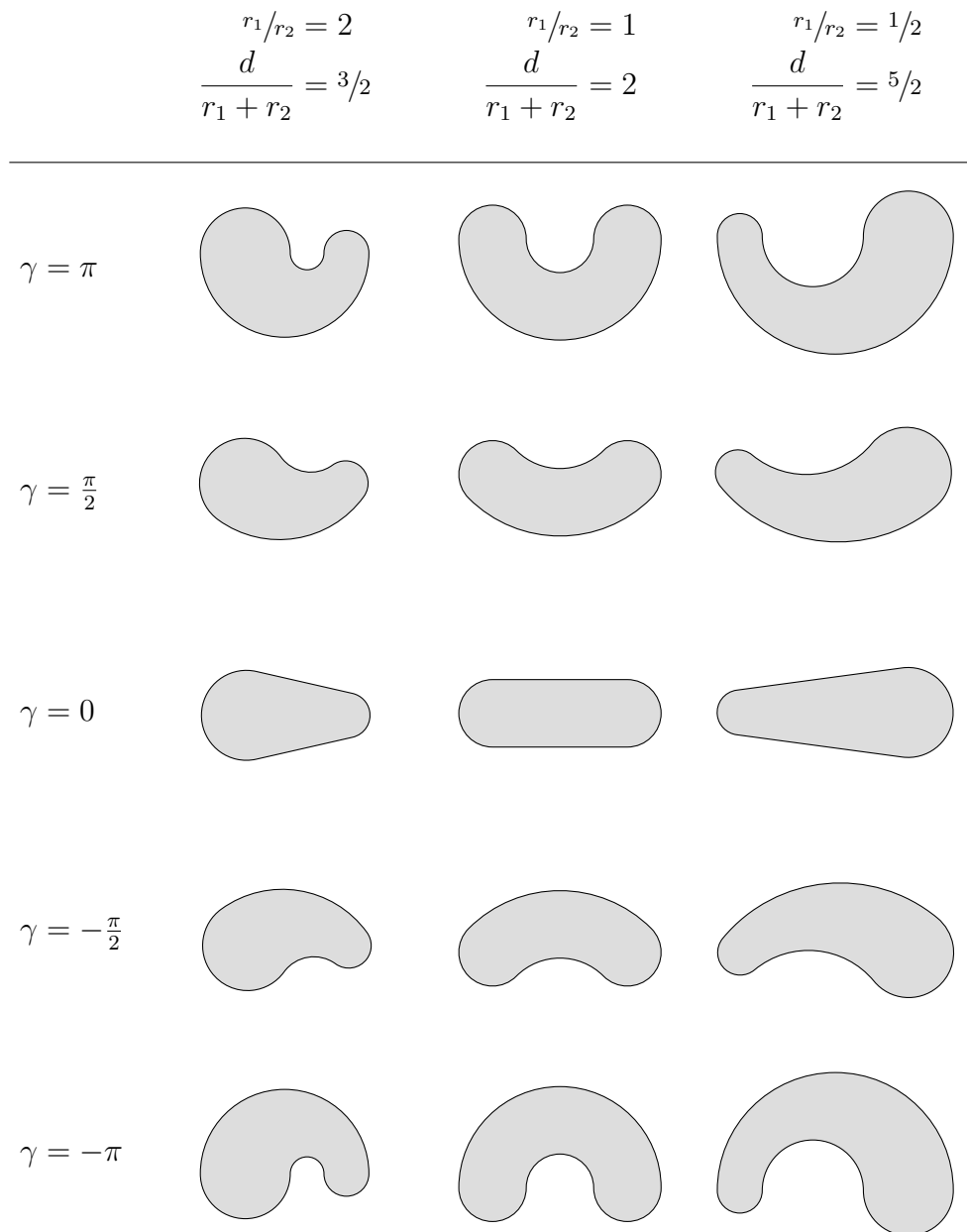


Abbildung 29: Die Matrix visualisiert die Möglichkeiten der parametrisierten CASHEW-Form.

Abbildungsverzeichnis

1	Rectbot mit der Pose $P = (30^\circ, -78^\circ)$	9
2	Rectbots Posenraum und vier Beispielposen	10
3	Rectbots eindimensionaler Phasenraum	13
4	Rectbots Bifurkationsdiagramm	14
5	Tablebot mit der Pose $P = (0, 4\pi/3, -\pi/3)$	15
6	Tablebots dreidimensionale Mannigfaltigkeit	16
7	Verschiedene Arten von Graphen	19
8	Schematische Übersicht eines Gestaltungsprozesses	21
9	Kreisbögen mit unterschiedlichen Krümmungen	23
10	Die CASHEW-Form	24
11	Polygondarstellung und Kreisbogendarstellung im Vergleich	26
12	Fehlerregion des UPDATEPOCS-Algorithmus	29
13	Zwei instabile Posen, die zueinander fallen	31
14	Eine gekrümmte Version von Tablebot	32
15	Eine schematische Darstellung der Kanten im Mannigfaltigkeitsgraphen	33
16	Drei von Hand eingestellte Schlüsselposen von Tablebot	39
17	Ein Teilbereich von Tablebots Mannigfaltigkeit	41
18	Fünfmal derselbe Konfigurationsraum, jeweils eingefärbt durch eine andere Fitnessfunktion	42
19	Algorithmisch ermittelte Schlüsselposen von Tablebot	42
20	Darstellung essentieller Einzelteile des LARA-Steckkits	45
21	Tablebot, realisiert durch das LARA-Steckkit	47
22	Sechs Schlüsselposen für Versuchsreihe 1.	49
23	Balkendiagramme der Ergebnisse aus Versuchsreihe 1	50
24	Vier diverse Morphologien für Versuchsreihe 2	51
25	Ausgewählte Standbilder aus Videoaufnahmen von Versuchsreihe 2	53
26	Drei verschiedene Kreisbögen und ihre Eigenschaften.	61
27	Drei Möglichkeiten, Ecken durch Kreisbögen zu erzeugen	65

28	CASHEW-Form für den Anhang	66
29	CASHEW-Form-Matrix	70

Tabellenverzeichnis

1	Koeffiziententabelle für die Fitnessfunktionen	40
2	Eigenschaften der einzelnen Körperteile von Tablebot	48
3	Fitnesskoeffiziententabelle für Versuchsreihe 1	50
4	Posenkombinationen für die Bewegungsabläufe	50
5	Mittelwert μ und Standardabweichung σ der verwendeten Normalverteilungen	52
6	Versuchsreihe 2: Strecke S	54
7	Versuchsreihe 2: Energie E	54
8	Versuchsreihe 2: Energieeffizienz S/E	54
9	Bogendefinitionen der CASHEW-Form bei $\gamma = 0$	67
10	Bogendefinitionen der CASHEW-Form bei $\gamma \neq 0$	69

Algorithmusverzeichnis

1	POSTURIZE Roboterstruktur $\mathcal{R}$ mit Winkelliste Φ	28
2	UPDATEPOCs durch Kreisbogenform $\mathcal{F}$	30
3	FINDENACHBAR von Knoten v in Dimension i und Richtung d	34
4	FINDEZUGEHÖRIGKEITEN in Knotenliste V	36
5	BESTEPOSE ab Startknoten v_0 mit Fitnessfunktion f	38
6	TRANSFORMIERE Form $\mathcal{F}$ um Winkel ψ und Versatz v	64
7	TIEFSTENPUNKTE von Form $\mathcal{F}$	64

Literatur

- [1] R. Pfeifer und J. Bongard, *How the body shapes the way we think: a new view of intelligence*. MIT press, 2006.
- [2] K. Sims, „Evolving 3D Morphology and Behavior by Competition,“ *Artificial Life*, 1994, ISSN: 1064-5462. DOI: 10.1162/artl.1994.1.4.353.
- [3] J. Bongard, „Morphological change in machines accelerates the evolution of robust behavior,“ *Proceedings of the National Academy of Sciences*, Jg. 108, Nr. 4, S. 1234–1239, 2011, ISSN: 0027-8424. DOI: 10.1073/pnas.1015390108.
- [4] N. Jakobi, P. Husbands und I. Harvey, „Noise and the reality gap: The use of simulation in evolutionary robotics,“ in *European Conference on Artificial Life*, Springer, 1995, S. 704–720.
- [5] L. Brodbeck, S. Hauser und F. Iida, „Morphological Evolution of Physical Robots through Model-Free Phenotype Development,“ *PLOS ONE*, Jg. 10, Nr. 6, S. 1–17, Juni 2015. DOI: 10.1371/journal.pone.0128444.
- [6] S. Höfer, M. Hild und M. Kubisch, „Using slow feature analysis to extract behavioural manifolds related to humanoid robot postures,“ in *Tenth International Conference on Epigenetic Robotics*, 2010, S. 43–50.
- [7] R. Peters und O. C. Jenkins, „Uncovering manifold structures in robonaut’s sensory-data state space,“ in *5th IEEE-RAS International Conference on Humanoid Robots, 2005.*, IEEE, 2005, S. 369–374.
- [8] M. Hild und M. Kubisch, „Self-exploration of autonomous robots using attractor-based behavior control and ABC-learning,“ in *Eleventh Scandinavian Conference on Artificial Intelligence*, IOS Press, 2011, S. 153–162.
- [9] S. Bethge, „ABC-Learning: Ein Lernverfahren zur modellfreien Selbstexploration autonomer Roboter,“ Diplomarbeit, Humboldt-Universität zu Berlin, Institut für Informatik, 2014.

- [10] M. Yim, W.-m. Shen, B. Salemi, D. Rus, M. Moll, H. Lipson, E. Klavins und G. S. Chirikjian, „Modular Self-Reconfigurable Robot Systems [Grand Challenges of Robotics],“ *IEEE Robotics Automation Magazine*, Jg. 14, Nr. 1, S. 43–52, 2007. DOI: 10.1109/MRA.2007.339623.
- [11] M. Yim, D. Duff und K. Roufas, „PolyBot: a modular reconfigurable robot,“ in *Proceedings 2000 ICRA. Millennium Conference. IEEE International Conference on Robotics and Automation. Symposia Proceedings (Cat. No.00CH37065)*, Bd. 1, 2000, 514–520 vol.1. DOI: 10.1109/ROBOT.2000.844106.
- [12] P. White, V. Zykov, J. C. Bongard, H. Lipson u. a., „Three dimensional stochastic reconfiguration of modular robots,“ in *Robotics: Science and Systems*, Citeseer, 2005, S. 161–168.
- [13] M. Hild, T. Siedel, C. Benckendorff, C. Thiele und M. Spranger, „Myon, a New Humanoid,“ in *Language Grounding in Robots*, L. Steels und M. Hild, Hrsg. Boston, MA: Springer US, 2012, S. 25–44, ISBN: 978-1-4614-3064-3. DOI: 10.1007/978-1-4614-3064-3_2.
- [14] H. S. Raffle, A. J. Parkes und H. Ishii, „Topobo: A Constructive Assembly System with Kinetic Memory,“ in *Proceedings of the SIGCHI Conference on Human Factors in Computing Systems*, Ser. CHI '04, Vienna, Austria: Association for Computing Machinery, 2004, S. 647–654, ISBN: 1581137028. DOI: 10.1145/985692.985774.
- [15] M. Hild, M. Kubisch und S. Höfer, „Using quadric-representing neurons (QRENs) for real-time learning of an implicit body model,“ in *Proceedings of the 11th Conference on Mobile Robot and Competitions*, 2011.
- [16] L. Smith und M. Gasser, „The development of embodied cognition: Six lessons from babies,“ *Artificial life*, Jg. 11, Nr. 1-2, S. 13–29, 2005.
- [17] A. Marina, „Die Relationen des Palaeencephalons (Edinger) sind nicht fix,“ *Neurol. Centralbl*, Jg. 34, S. 338, 1915.